

William Stallings
Computer Organization
and Architecture
10th Edition

Chapter 14
CPU Structure and Function

Key points

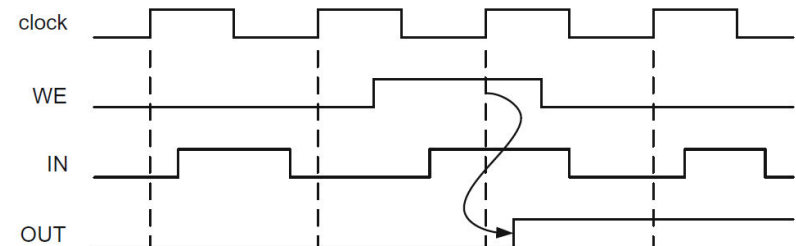
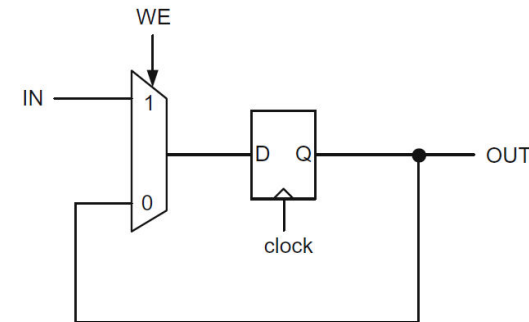
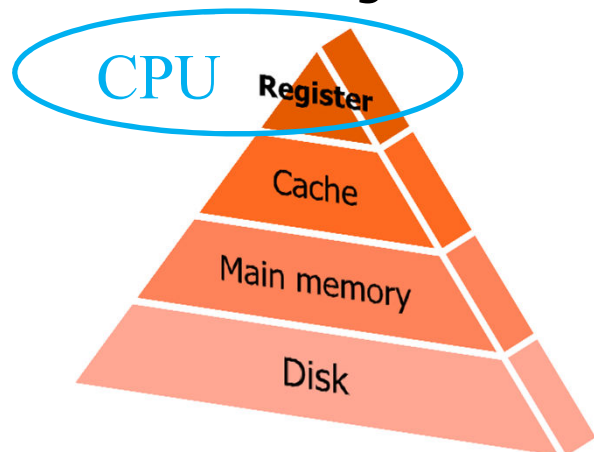
- User-visible **registers** and control/status registers

- User-visible registers

- General purpose
- Special use: e.g., fixed point num, floating point num, address, index, segment pointer

- Control/status registers, e.g.

- Program counter (PC)
- Program status word (PSW)



One-bit register and a sample timing diagram

Key points

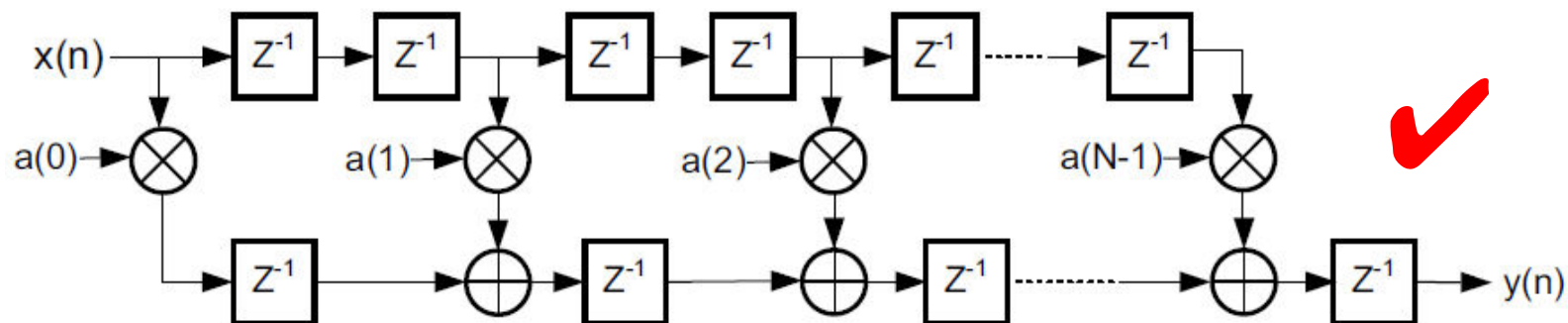
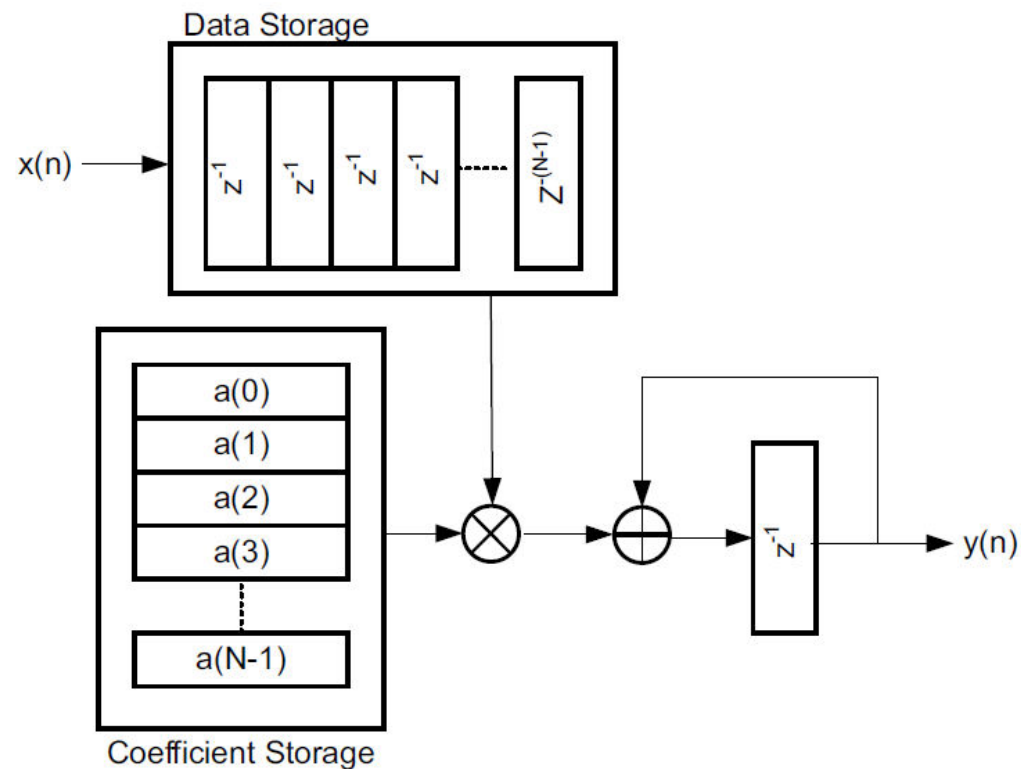
- **Pipeline**

- Each stage works on different instruction at the same time
- Branches complicate the design



- FIR filter
- Pipelined form?

$$y(k) = \sum_{n=0}^{N-1} a(n)x(k-n) \quad k = 0, 1, \dots$$

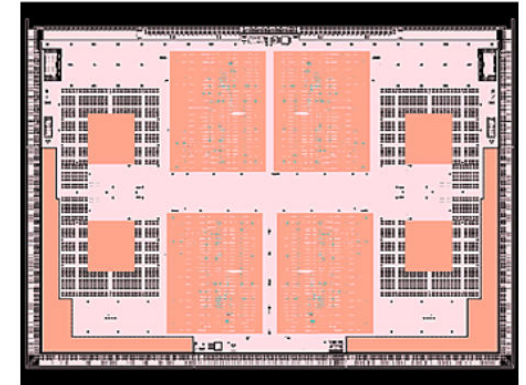


14.1 Processor organization

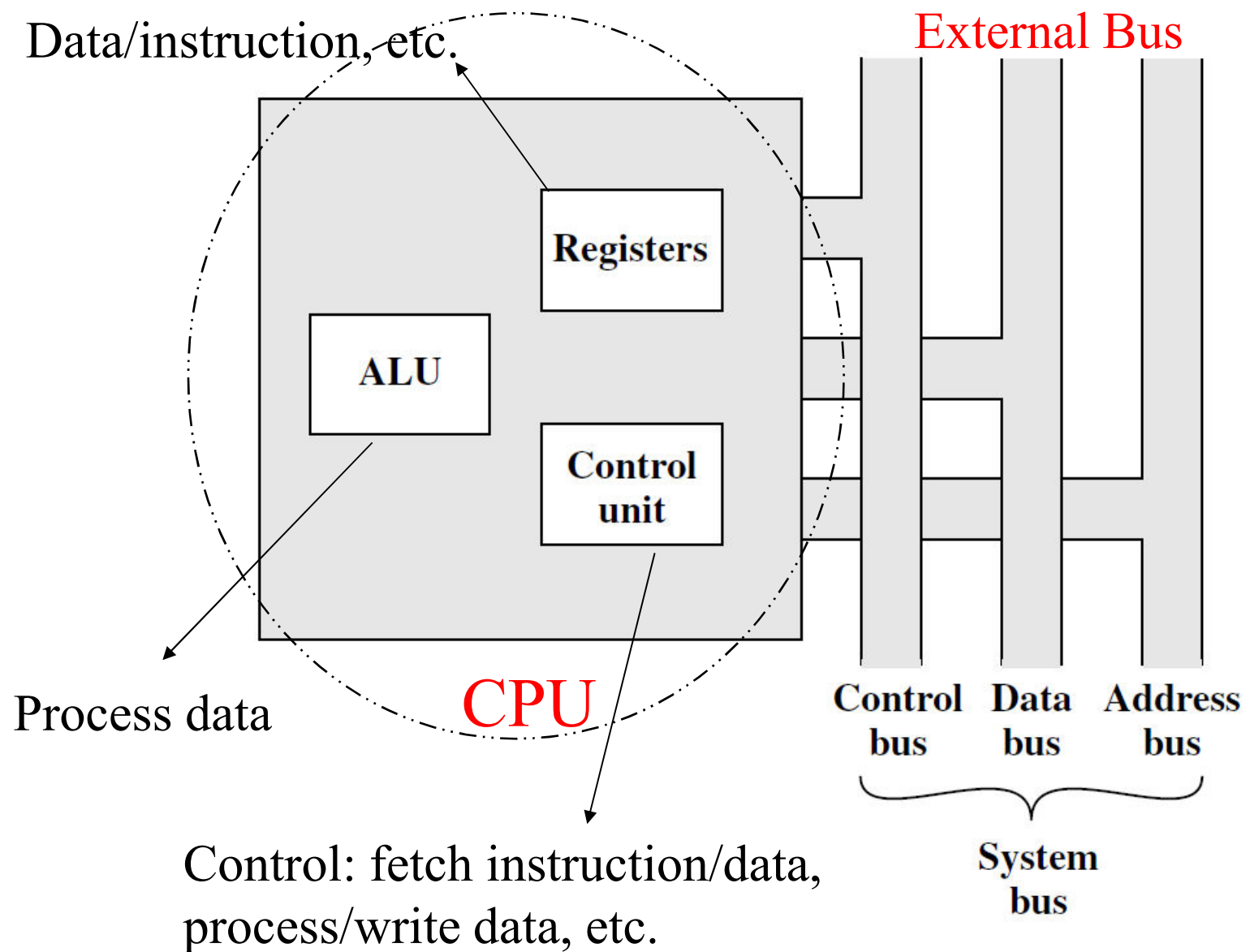
- CPU must do:
 - Fetch instruction
 - Interpret instruction
 - Fetch data
 - Process data
 - Write data
- CPU needs a *small internal memory*
 - Store *data* temporarily
 - Locate next *instruction* (*address*)
 - Store *instructions* temporarily



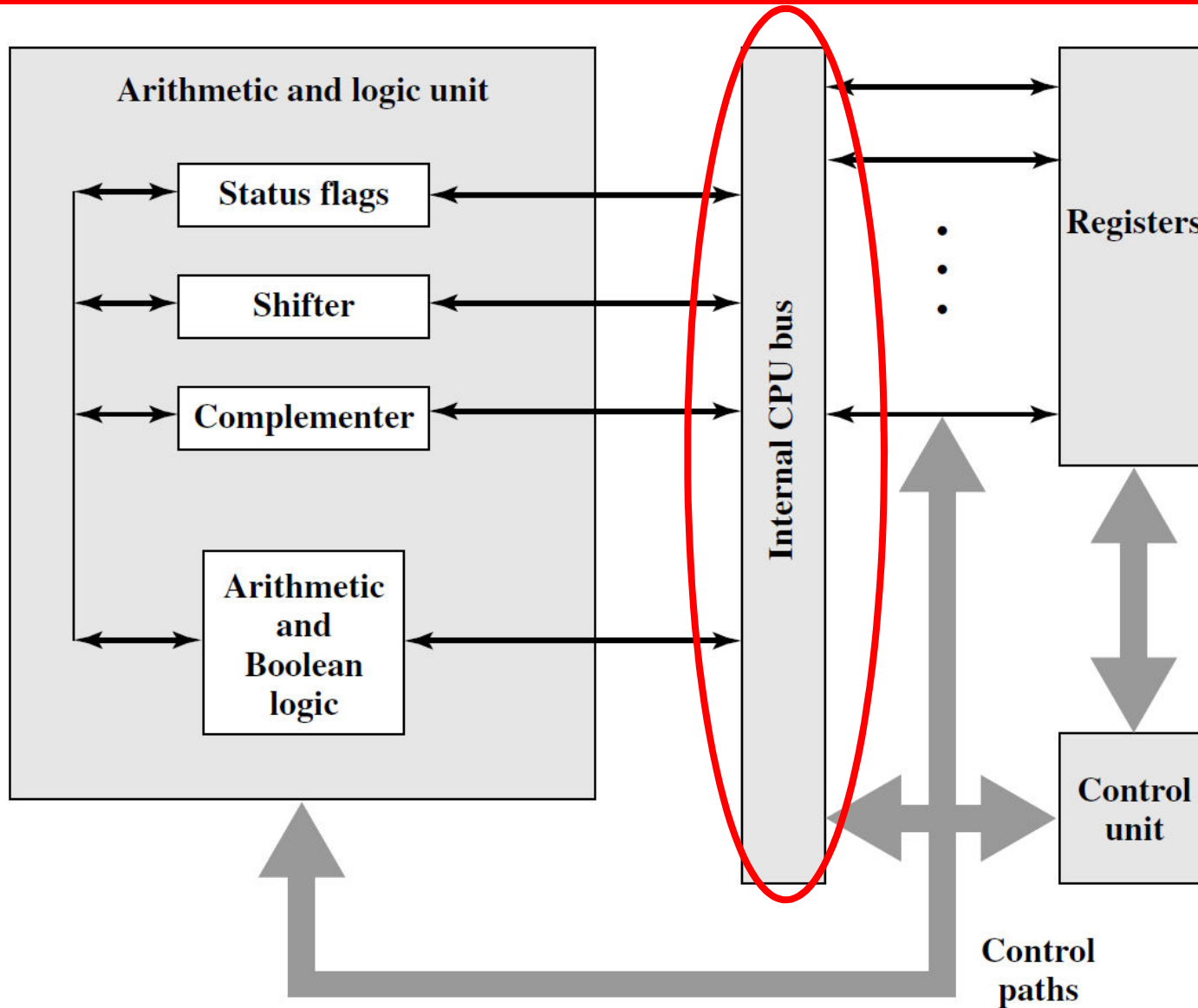
28nm 4-core



CPU With Systems Bus



CPU Internal Structure

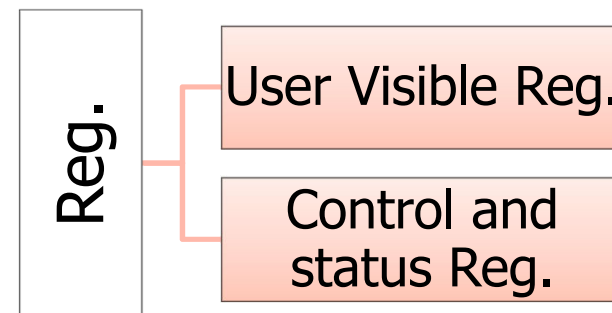
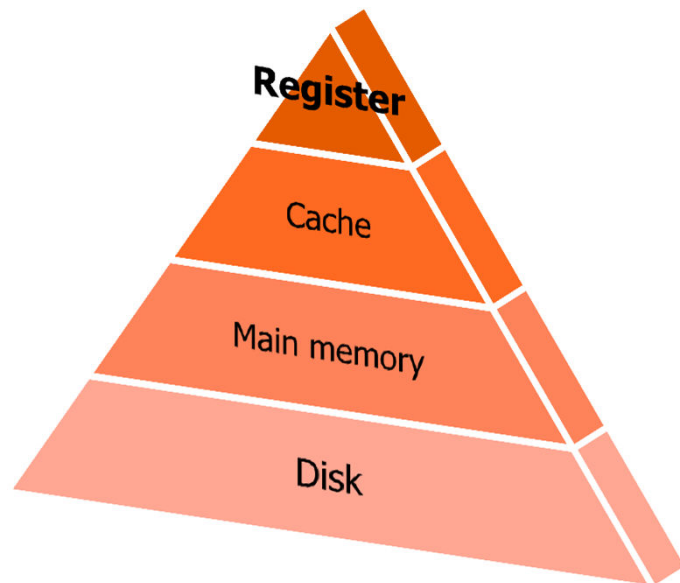


14.2 Register organization

- User-visible registers (*To whom? Why?*)
- Control and status registers (*Why?*)

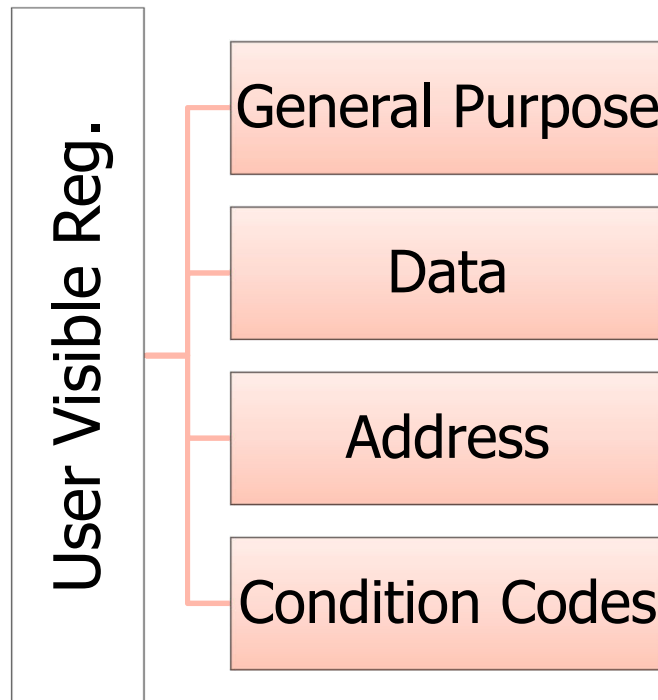
Registers

- Top level of *memory hierarchy*
- Temporary storage for CPU
- *Number* and *function* vary between processor designs
- Two categories: user-visible Reg. & control and status Reg.



User Visible Registers

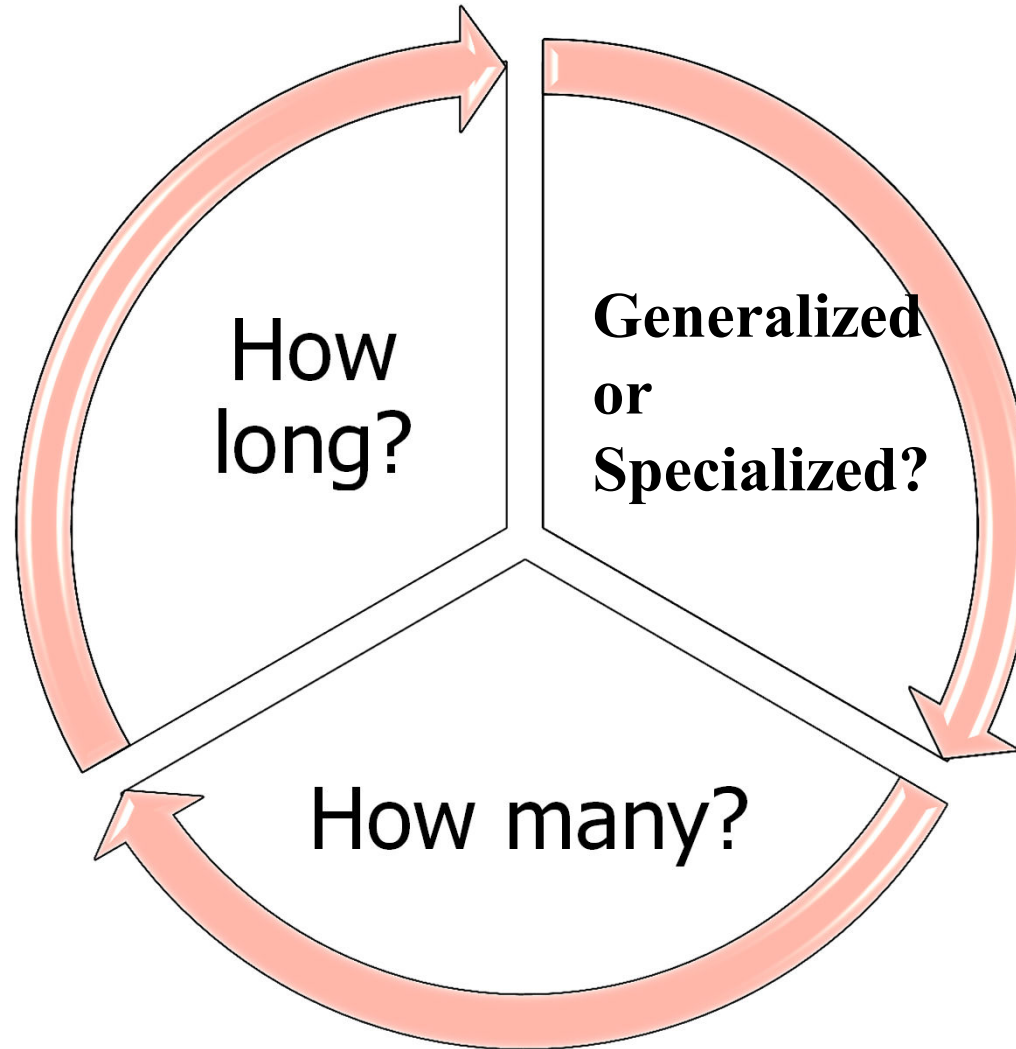
- A **user-visible register** is one that may be referenced by means of the machine language.



General Purpose /Data /Address Reg

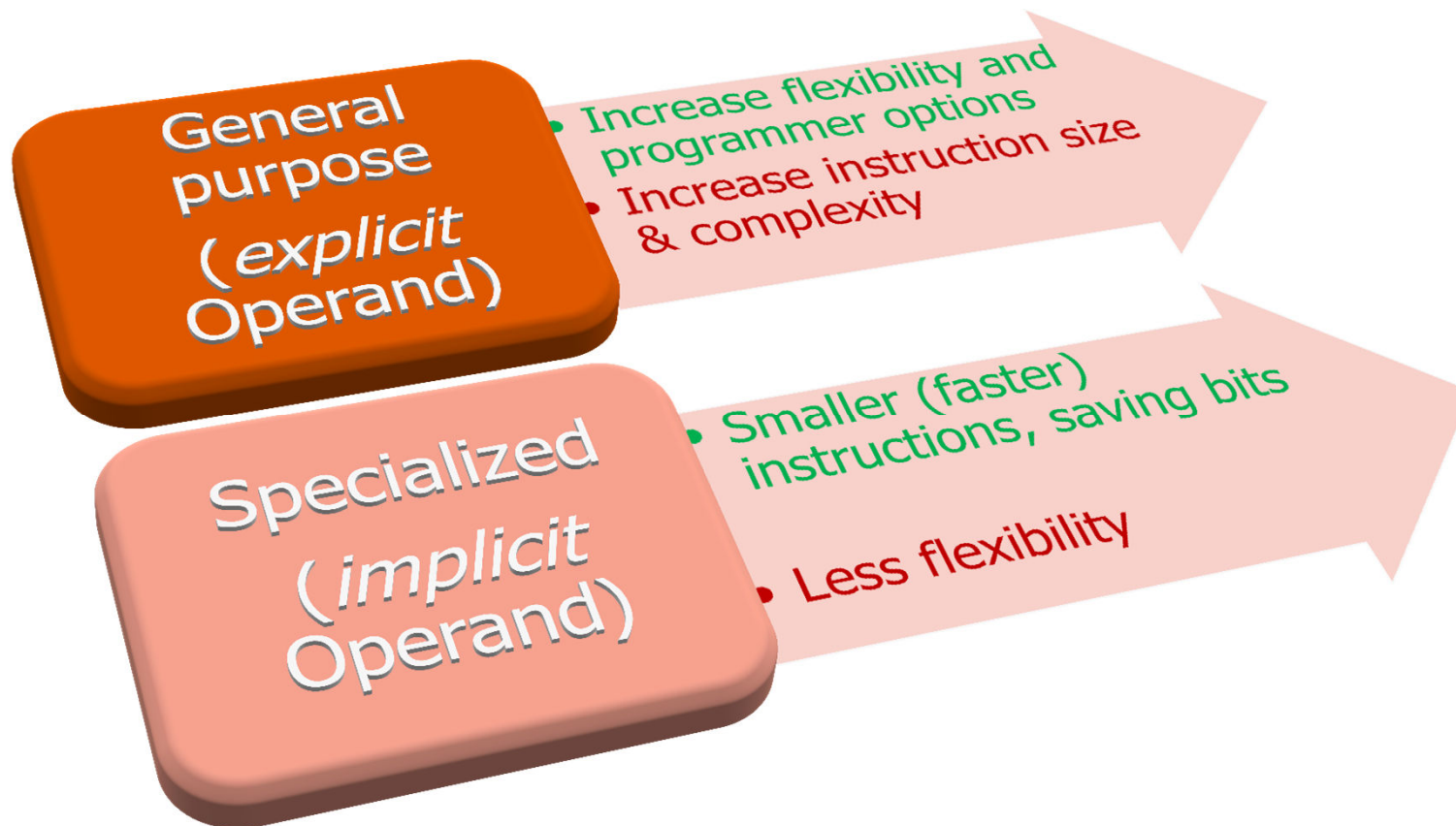
- **General Purpose** Registers
 - True general purpose: *orthogonal to operation* (*any Opcode to any Operand*)
 - Restricted e.g. dedicated for *floating-point* and *stack* operations
- **Data** registers: only hold data.
- **Address** registers
 - General address
 - Segment pointers
 - Index registers
 - Stack pointer

Some Design Issues to Be Addressed



Generalized or specialized

- Making them generalized or specialized affects instruction set design



How Many GP Registers?

- *Reg No.*  *Instruction set*
 - more registers require more operand specifier bits
 - 8 ~32 registers appear optimum
- Fewer  more memory references
 - Fewer register (indirect) addressing modes
- More *does not* reduce memory references and takes up processor real estate
- A new approach(ch 15): RISC with hundreds of registers

How long?

- Long enough to hold full *address*
- Long enough to hold full *word*
- Often possible to combine two data registers
 - C++ programming
 - int a; (32bits, 4bytes)
 - double int a; (64bits, 8bytes)
 - long double a; (96bits, 12bytes)

Condition Code Registers

- Sets of individual bits
 - e.g. result of last operation was zero, positive, negative, or overflow
- Can be read (implicitly) by programs
 - e.g. Jump if zero
- Can not (usually) be set by programs

Condition Code Registers

Advantages	Disadvantages
1. Because condition codes are set by normal arithmetic and data movement instructions, they should <u>reduce the number of COMPARE and TEST instructions needed</u>. 2. Conditional instructions, such as <u>BRANCH</u> are <u>simplified relative to composite instructions</u>, such as TEST and BRANCH. 3. Condition codes facilitate <u>multiway branches</u>. For example, a TEST instruction can be followed by two branches, one on less than or equal to zero and one on greater than zero. 4. <u>Condition codes can be saved on the stack</u> during subroutine calls along with other register information.	1. <u>Condition codes add complexity</u>, both to the hardware and software. Condition code bits are often modified in different ways by different instructions, making life more difficult for both the microprogrammer and compiler writer. 2. Condition codes are irregular; they are typically not part of the main data path, so they require <u>extra hardware connections</u>. 3. Often condition code machines must <u>add special non-condition-code instructions</u> for special situations anyway, such as bit checking, loop control, and atomic semaphore operations. 4. In a pipelined implementation, condition codes <u>require special synchronization</u> to avoid conflicts.

Register save and restore

- For *call & return* procedure,
When "Call": automatic saving
When "Return": automatic restoring
- Others,
Saving and restoring by *user*

Control & Status Registers

- Most are **not visible** to the user.
- Program Counter(PC)
 - Always *points to* the **next** instruction
- Instruction Register(IR)
 - Contains *instruction*, to decode and analyze
- Memory Address Register(MAR)
 - Contains *memory address*, and connects to address bus
- Memory Buffer Register(MBR)
 - Contains *data*, and connects to data bus

Program Status Word

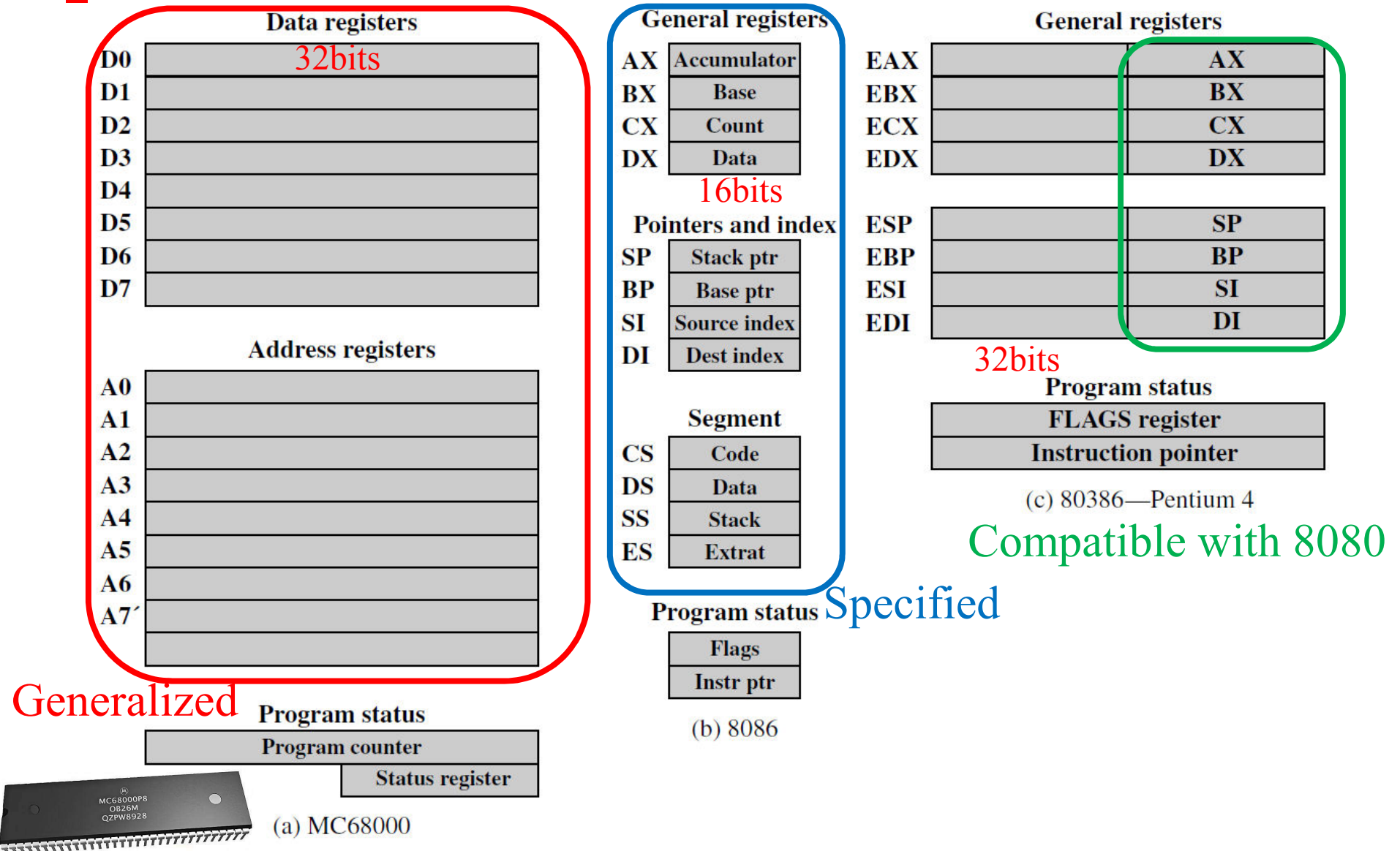
PSW include the following:

- Includes *Condition Codes*
 - Sign of last result/Zero/Carry/Equal/Overflow
- Interrupt enable/disable
- Supervisor
 - Allows *privileged instructions* to execute
 - Used by operating system
 - Not available to user programs

Other Registers

- May have registers pointing to:
 - Process control blocks (see O/S)
 - Interrupt Vectors (see O/S)
- CPU design and operating system design are closely linked (*compatibility*)

Example Register Organizations



14.3 Instruction Cycle

- Fetch
- Execute
- Interrupt

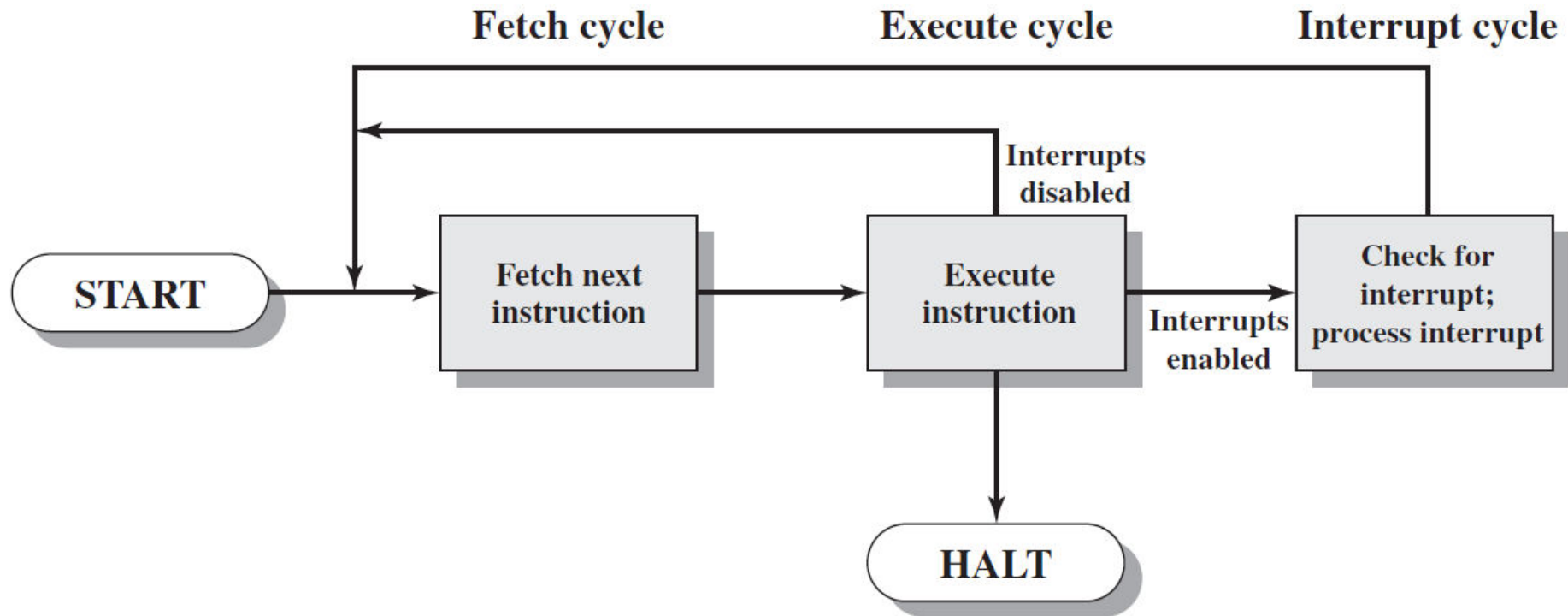
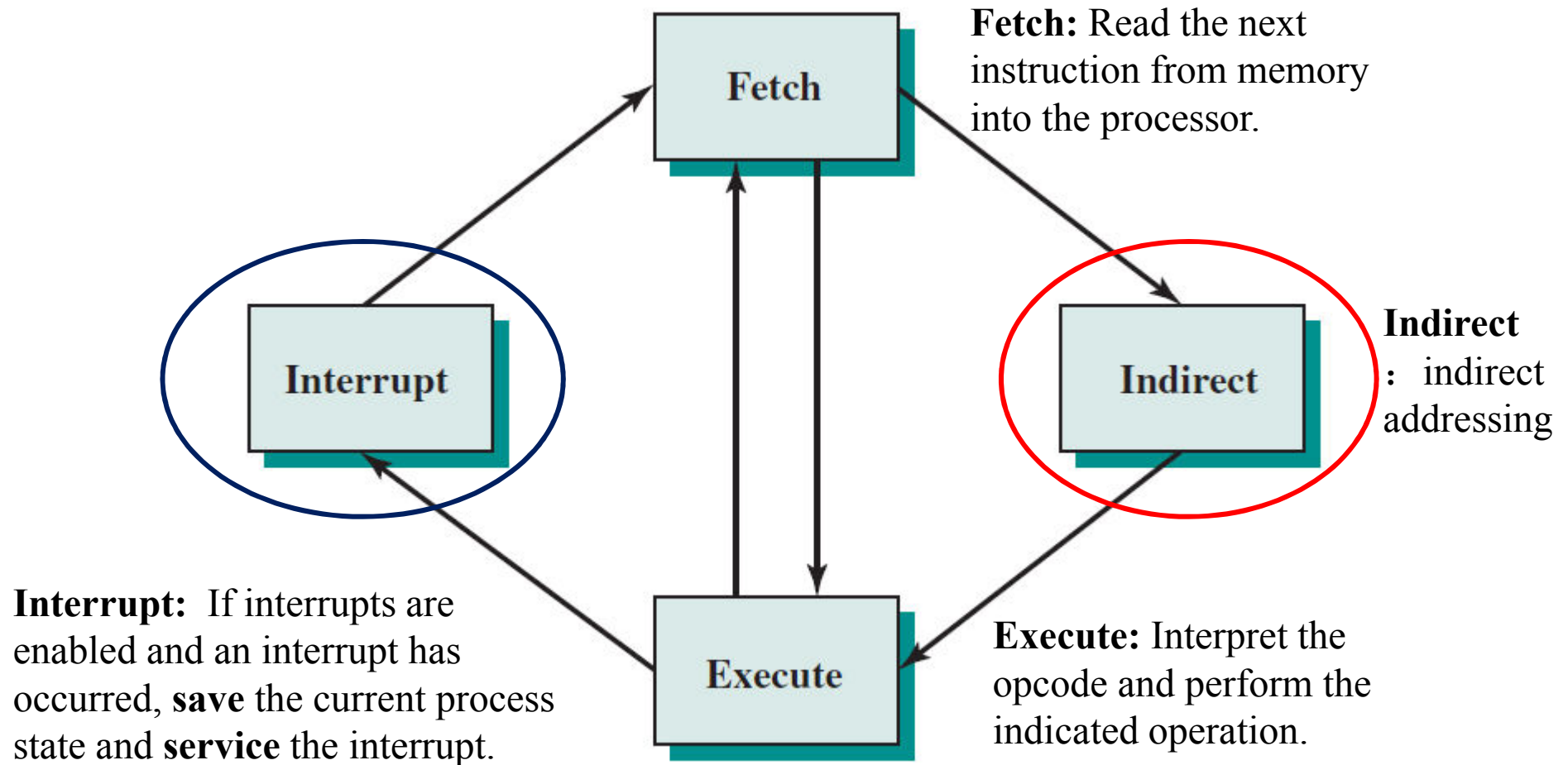


Figure 3.9 Instruction Cycle with Interrupts

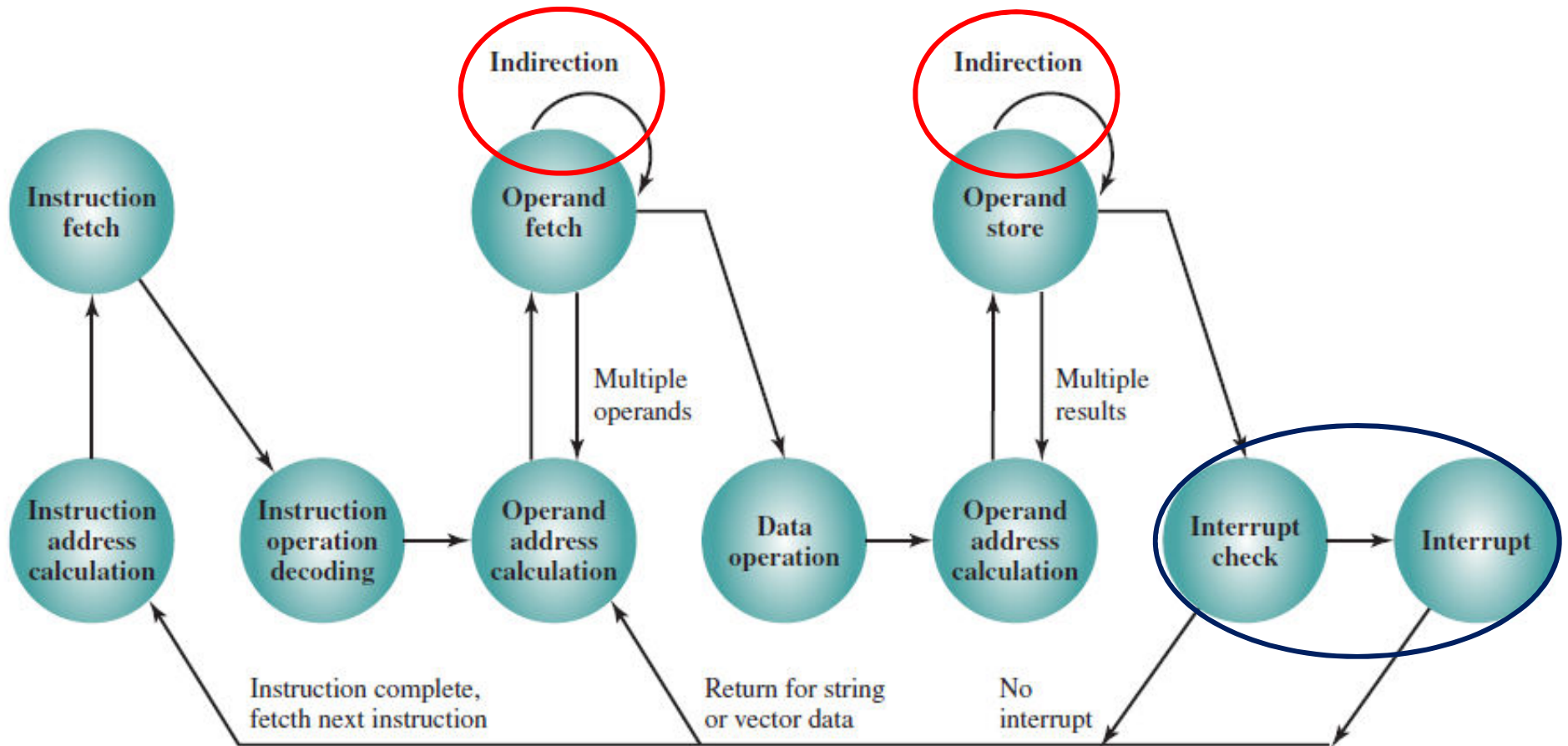
Indirect Cycle

- May require memory access to fetch operands
- Indirect addressing requires more memory accesses
- Can be thought of as *additional instruction subcycle*

Instruction Cycle with Indirect



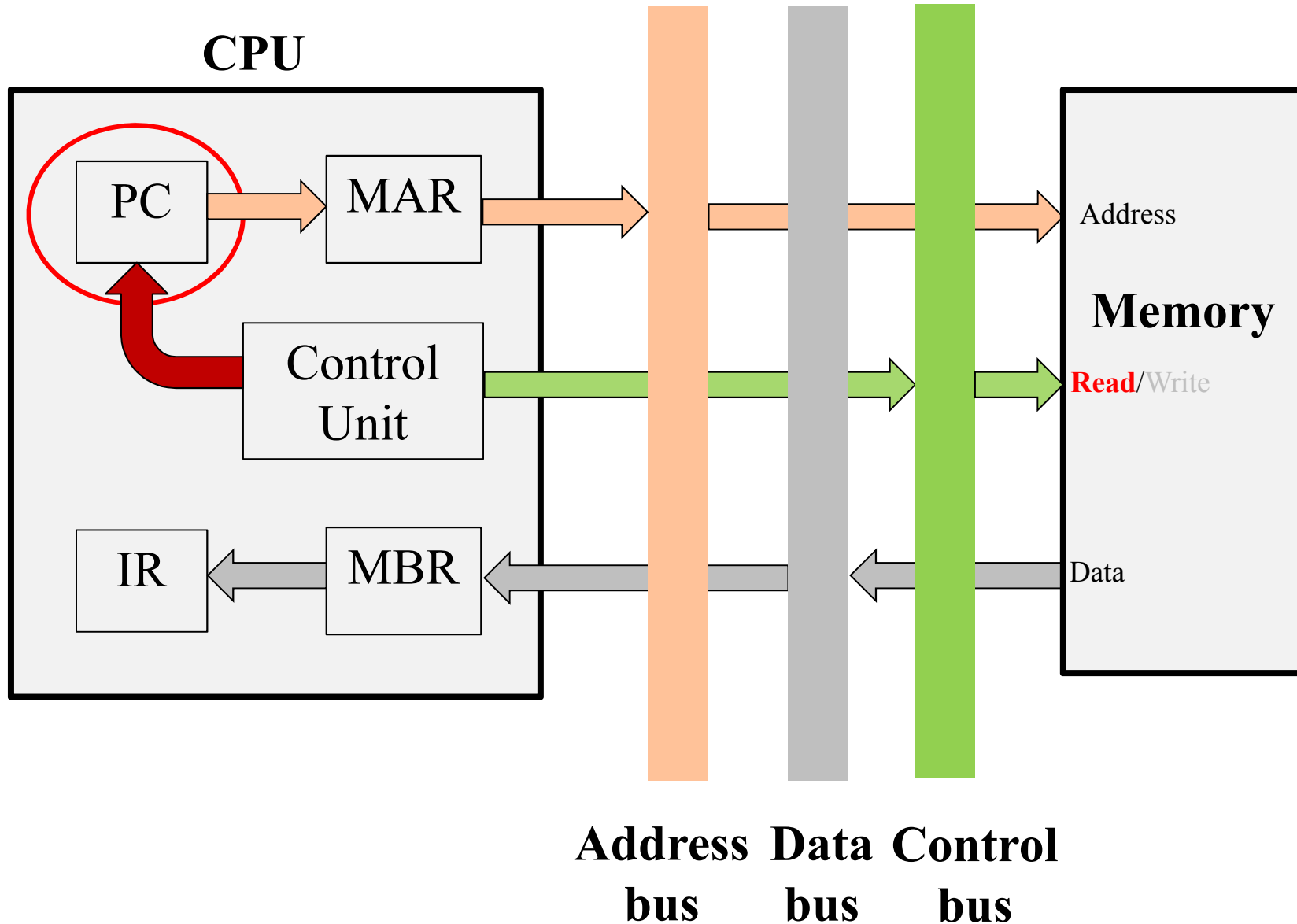
Instruction Cycle State Diagram



Data Flow (Instruction Fetch)

- Depends on *CPU design*
- In general: the followings must happen
- Fetch Instruction
 - *PC* contains address of next instruction
 - Address moved to *MAR*
 - Address placed on address bus
 - Control unit requests memory read
 - Result placed on data bus, copied to *MBR*, then to *IR*
 - Meanwhile *PC* incremented by 1

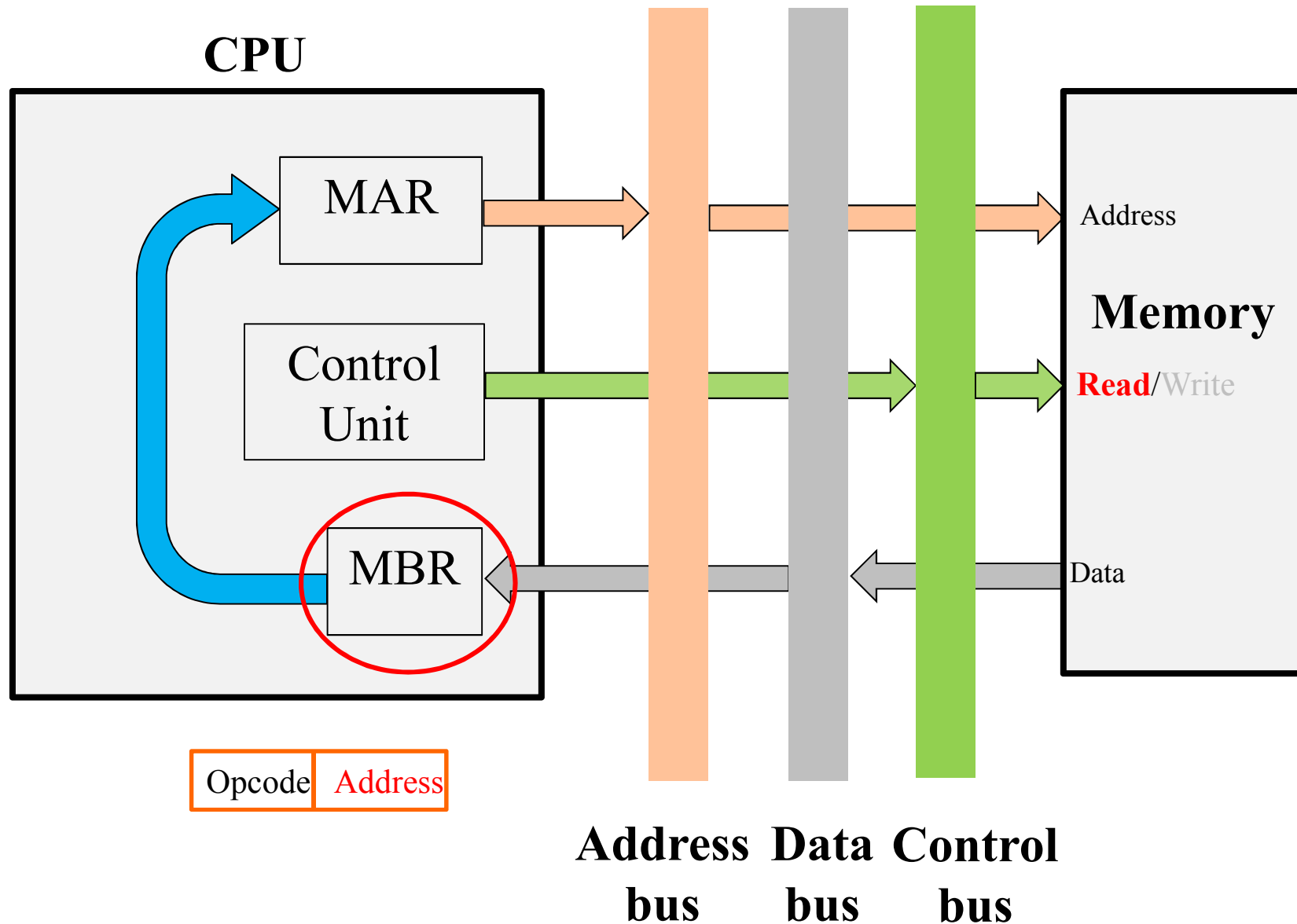
Data Flow (Instruction Fetch)



Data Flow (Indirect)

- *IR* is examined
- If indirect addressing, *indirect cycle* is performed
 - *Right most N bits* (*why?*) of **MBR** transferred to **MAR**
 - *Control unit* requests memory read
 - Result (address of operand) moved to **MBR**

Data Flow (Indirect Diagram)



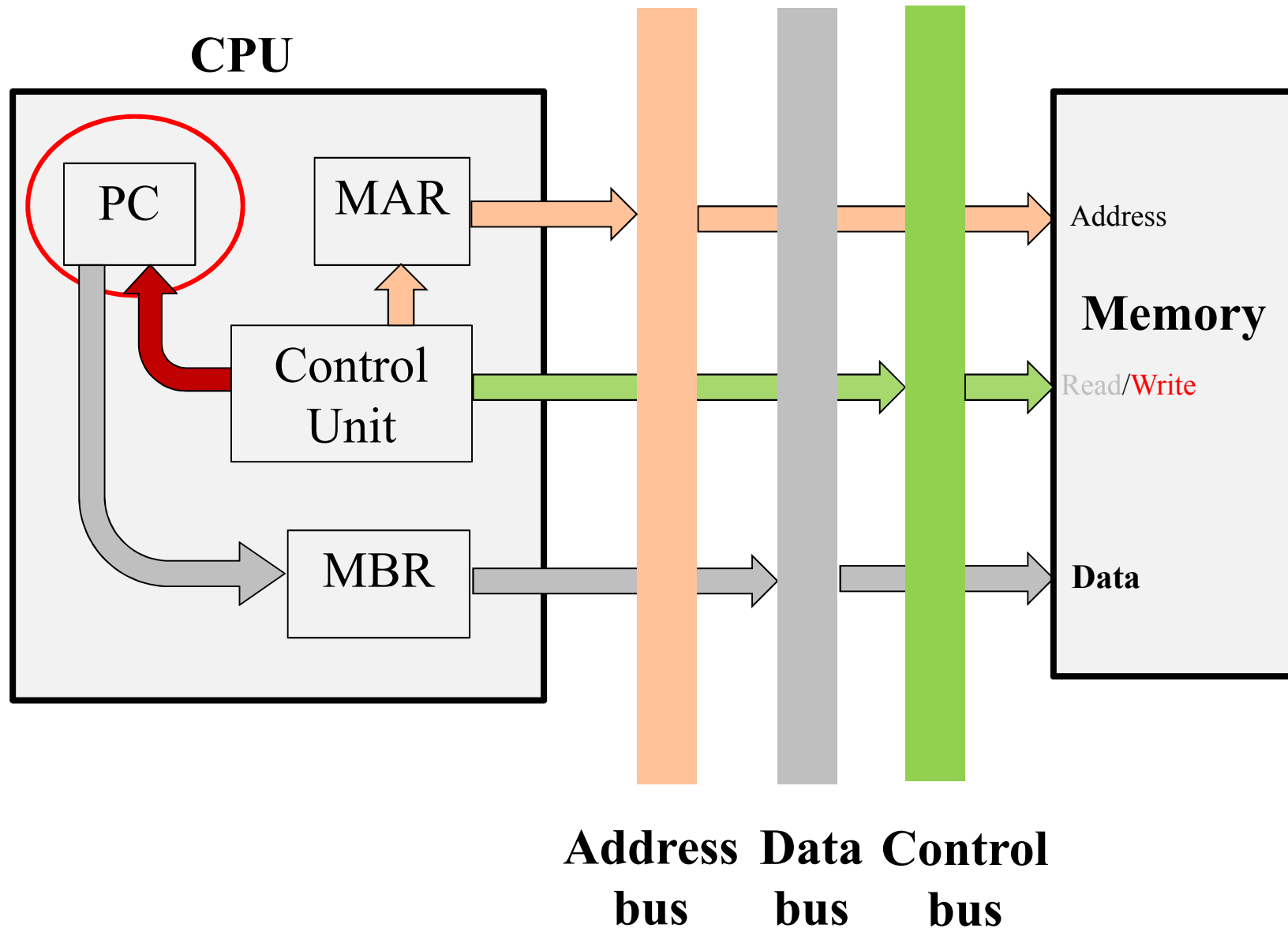
Data Flow (Execute)

- May take many forms
- Depends on instruction being executed
- May include
 - Memory read/write
 - Input/Output
 - Register transfers
 - ALU operations

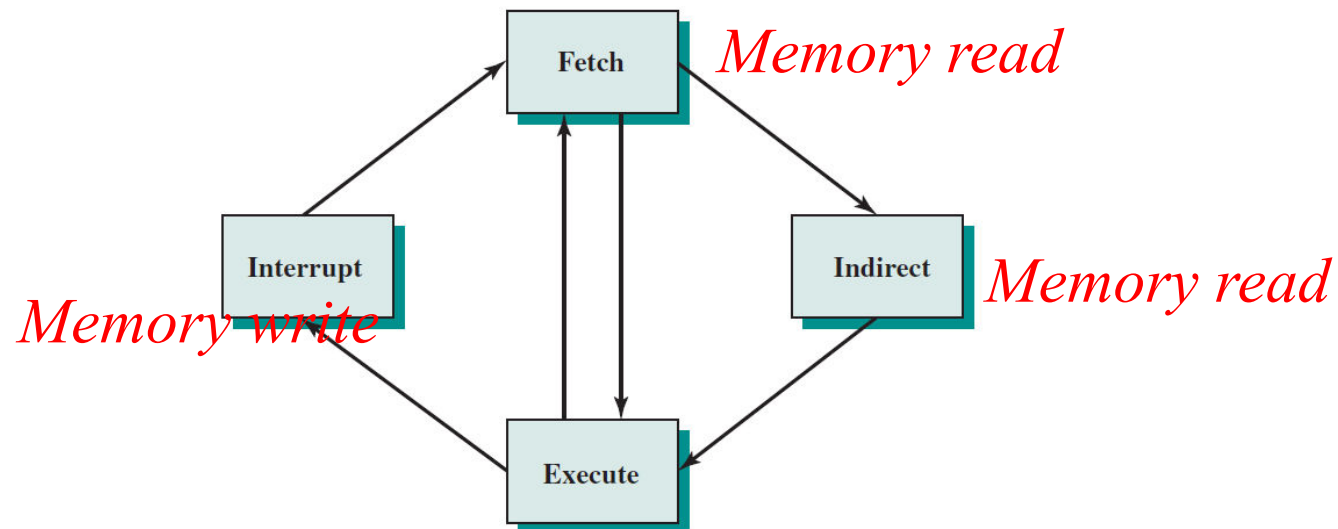
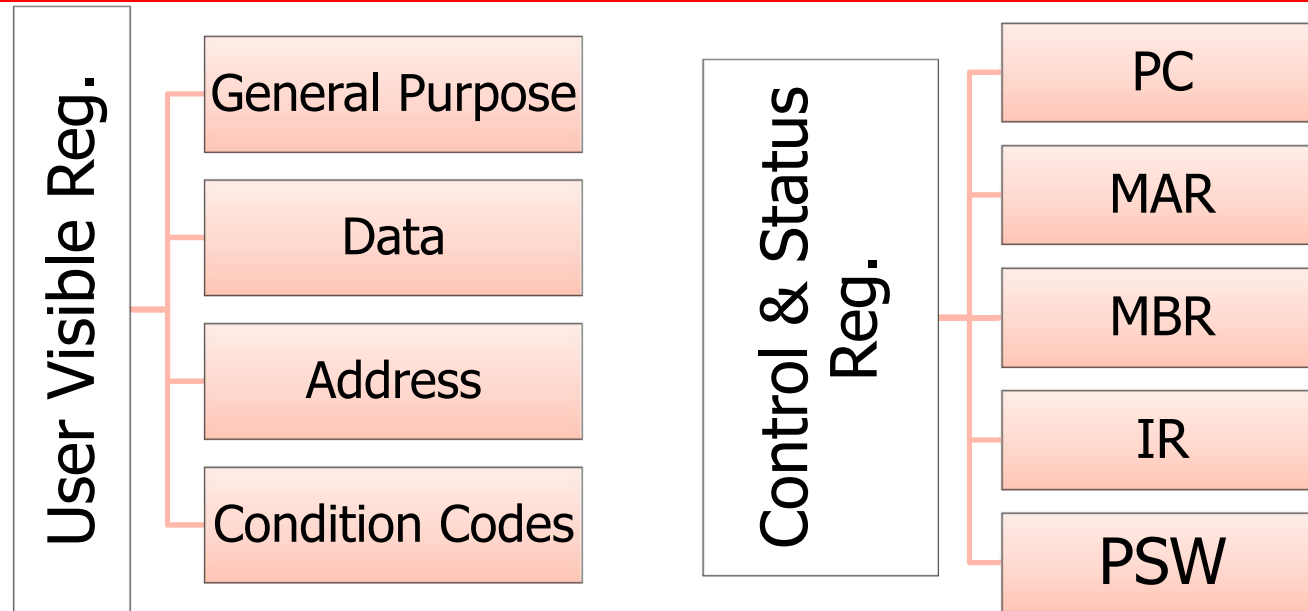
Data Flow (Interrupt)

- Current PC saved to allow resumption after interrupt
- Contents of *PC copied to MBR*
- *Special memory location* (e.g. *stack pointer*) loaded to MAR
- MBR written to *memory*
- PC loaded with address of *interrupt handling routine*
- Next instruction (first of interrupt handler) can be fetched

Data Flow (Interrupt Diagram)



Review



14.4 Instruction pipelining

- Pipelining strategy
- Pipeline performance
- Dealing with branches

Pipelining strategy

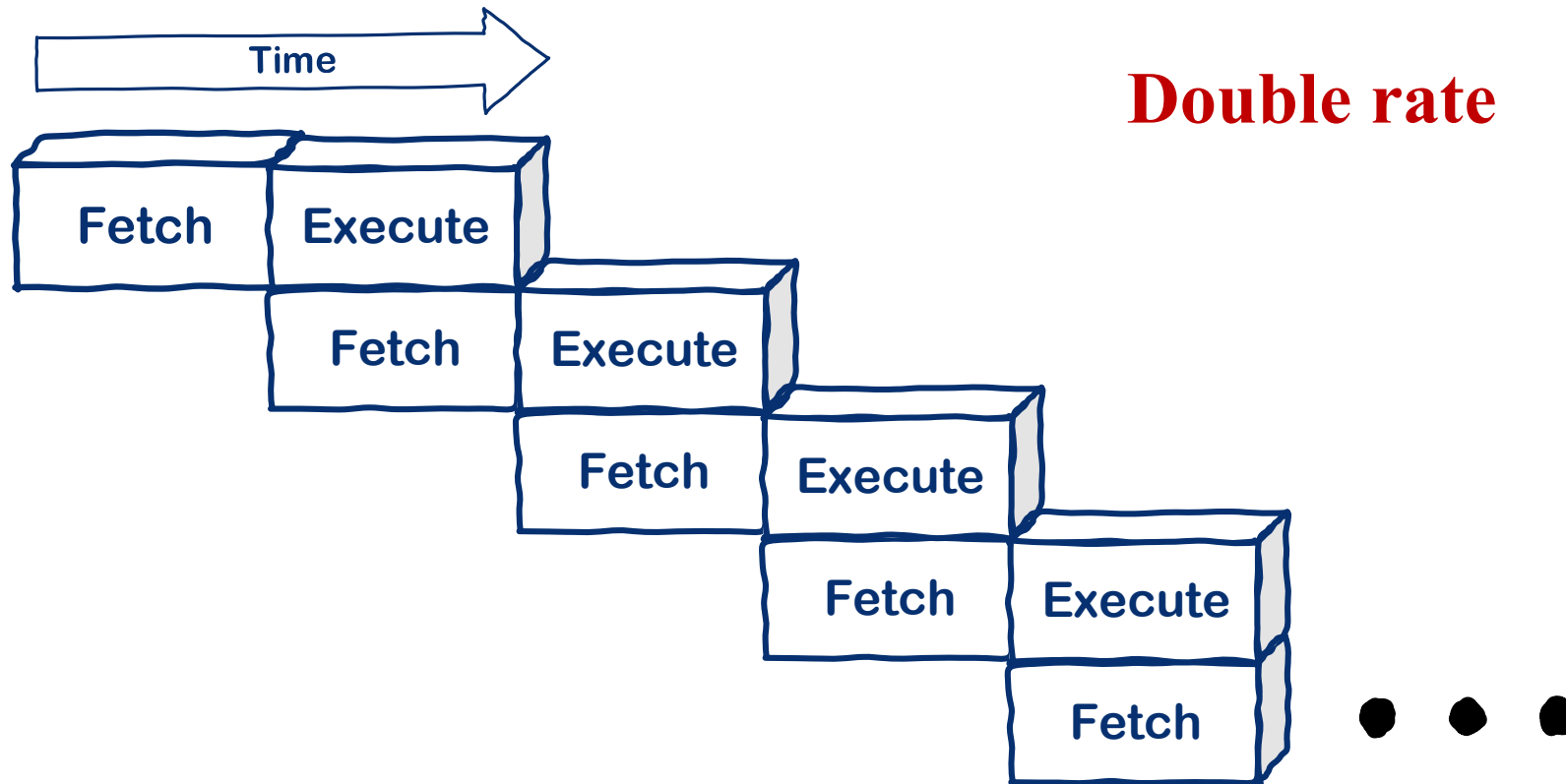
- Similar to assembly line
- Stages work simultaneously



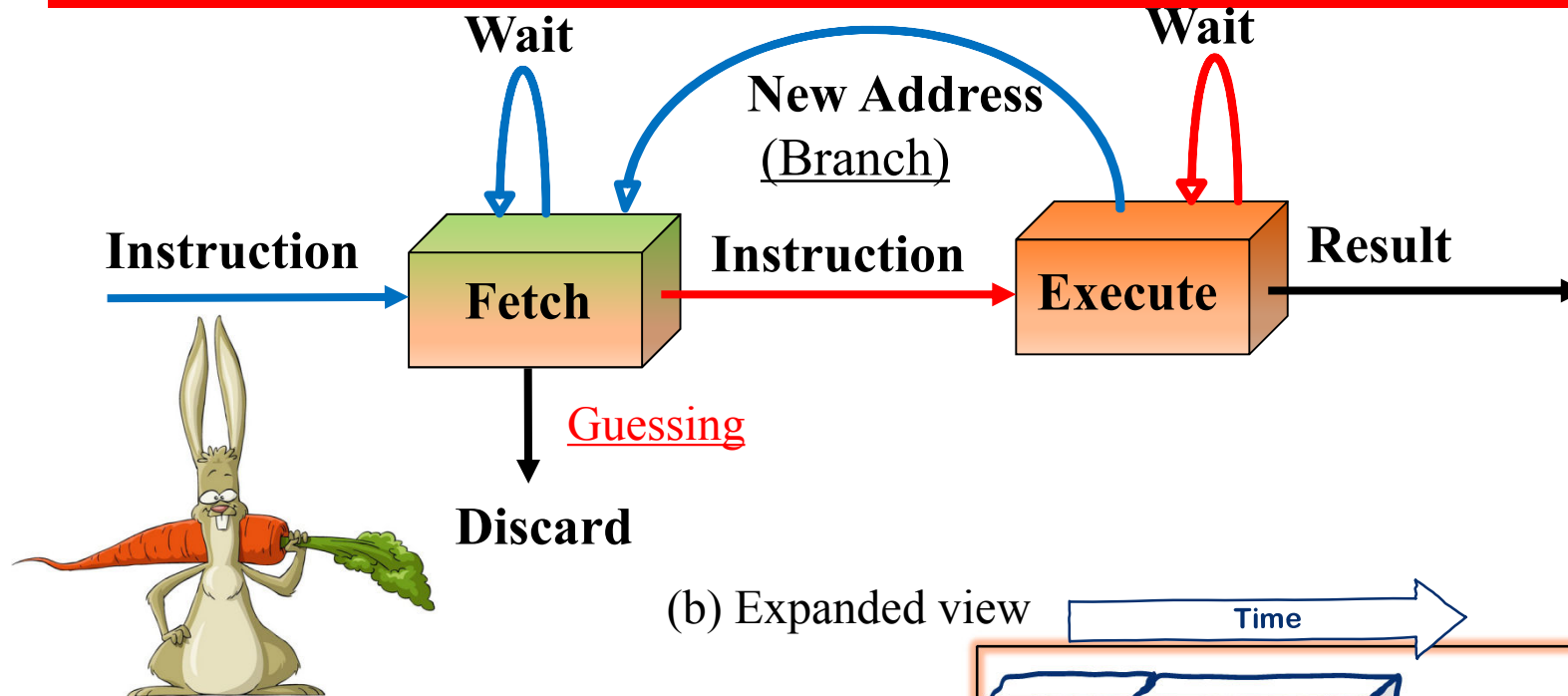
Two Stage Instruction Pipeline



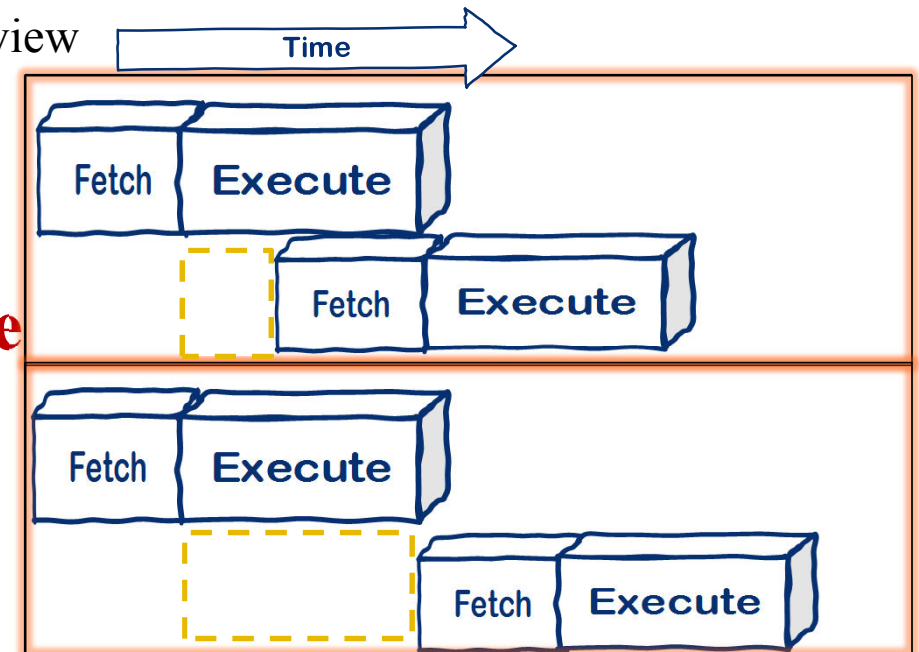
(a) Simplified view



Two Stage Instruction Pipeline



(b) Expanded view



2 Reasons { **Unequal stage time**
Branch

Decomposition of instruction processing

- Fetch instruction (FI)
- Decode instruction (DI)
- Calculate operands (CO)
- Fetch operands (FO)
- Execute instructions (EI)
- Write operand (WO)

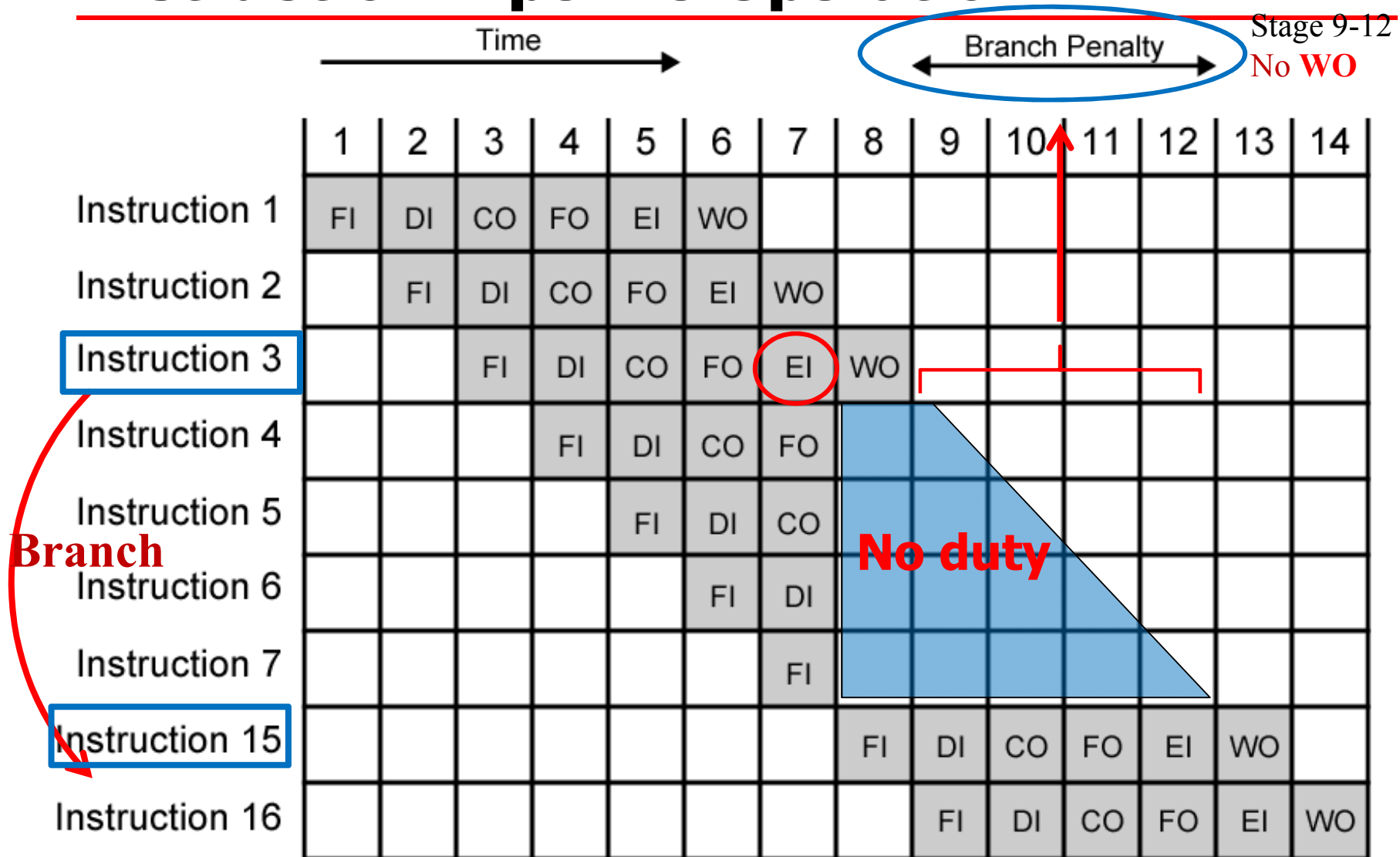
Timing Diagram for Instruction Pipeline Operation

Time
→

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

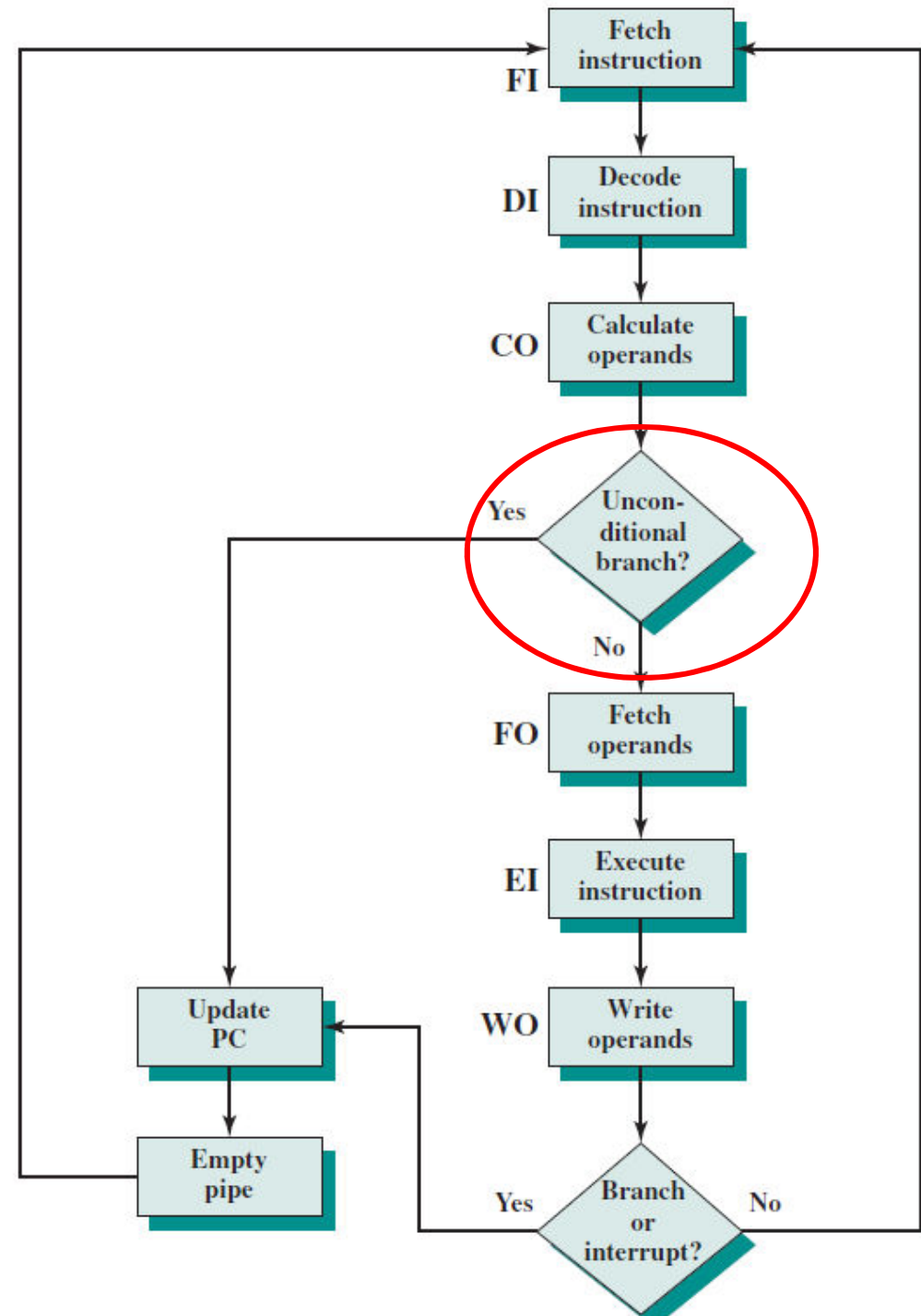
Assumption: 1) all six stages; 2) equal duration; 3) no memory conflicts

The Effect of a Conditional Branch on Instruction Pipeline Operation



Limit performance: 1) unequal duration; 2) conditional branch & interrupt 3) sequential instructions are logically dependent

Six Stage Instruction Pipeline



Alternative Pipeline Depiction

Time ↓

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I8	I7	I6	I5	I4	I3
9	I9	I8	I7	I6	I5	I4
10		I9	I8	I7	I6	I5
11			I9	I8	I7	I6
12				I9	I8	I7
13					I9	I8
14						I9

(a) No branches

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I15	I16	I15	I14	I13	I12
9	I16	I15	I14	I13	I12	I11
10		I16	I15	I14	I13	I12
11			I16	I15	I14	I13
12				I16	I15	I14
13					I16	I15
14						I16

(b) With conditional branch

Full @time 6
&7

I4~I7 are
flushed from
the pipeline

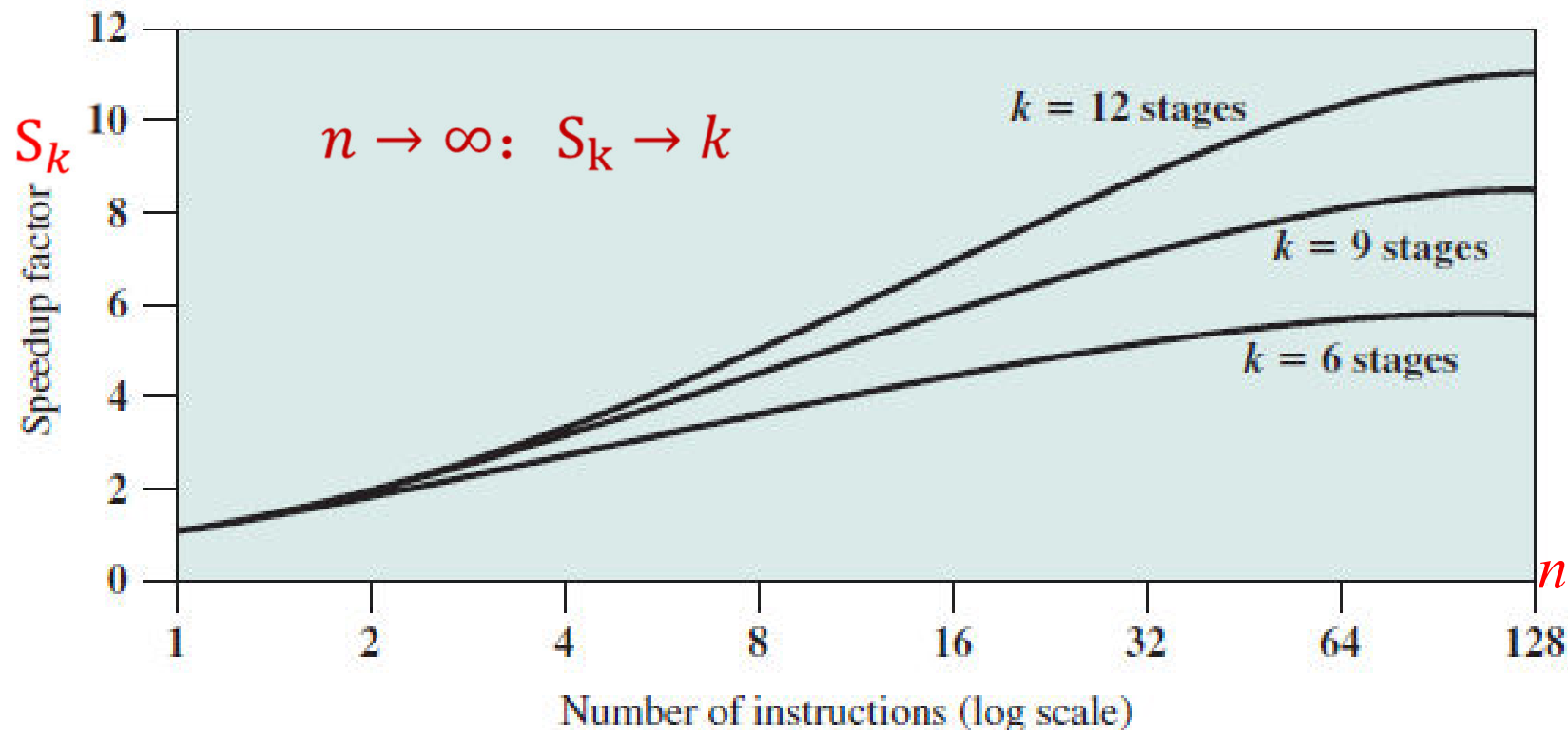
Wash-out

Pipeline Performance

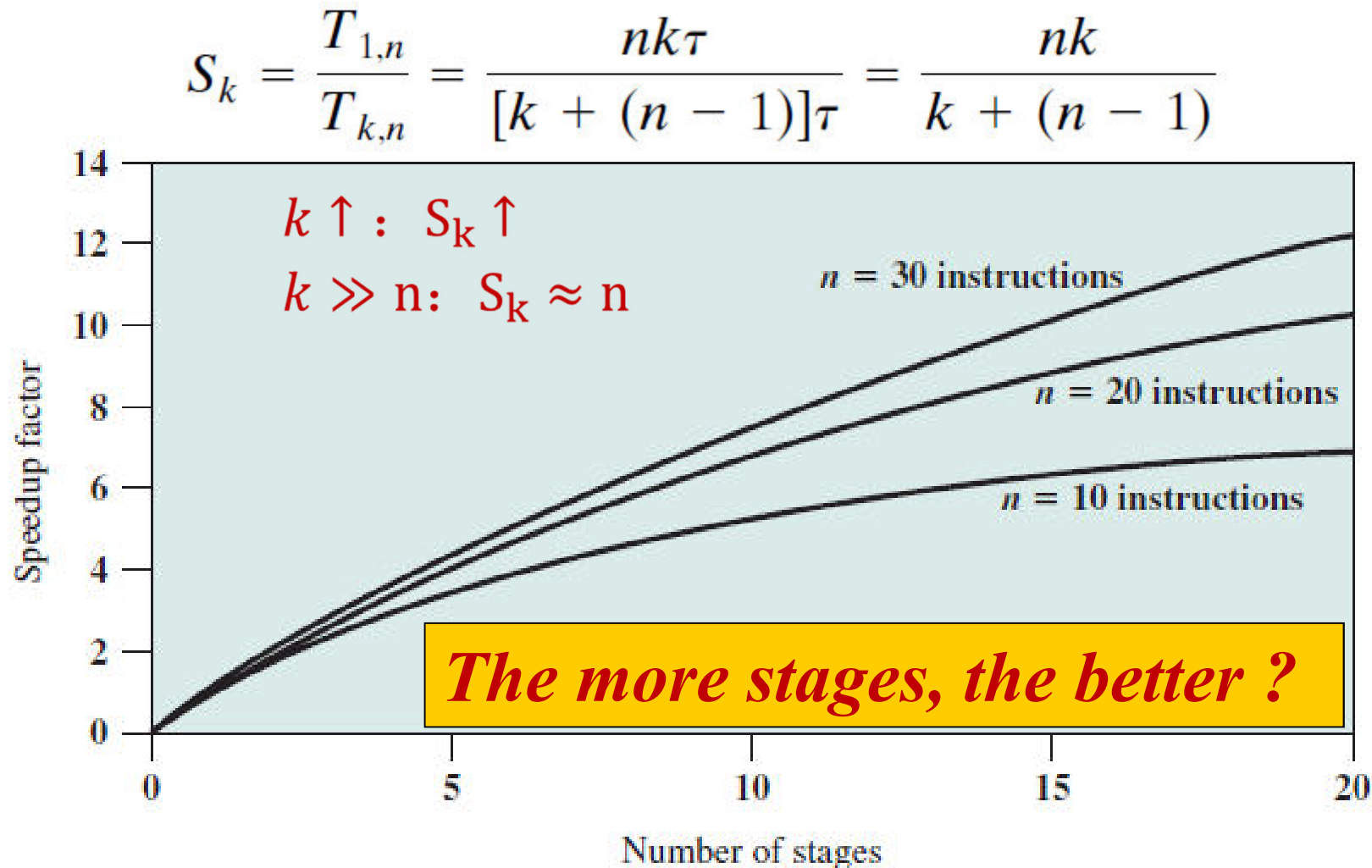
k stages to execute n instructions, no branch

$$T_{k,n} = [k + (n - 1)]\tau \rightarrow \text{one stage cycle time}$$

$$S_k = \frac{T_{1,n}}{T_{k,n}} = \frac{nk\tau}{[k + (n - 1)]\tau} = \frac{nk}{k + (n - 1)}$$



Pipeline Performance



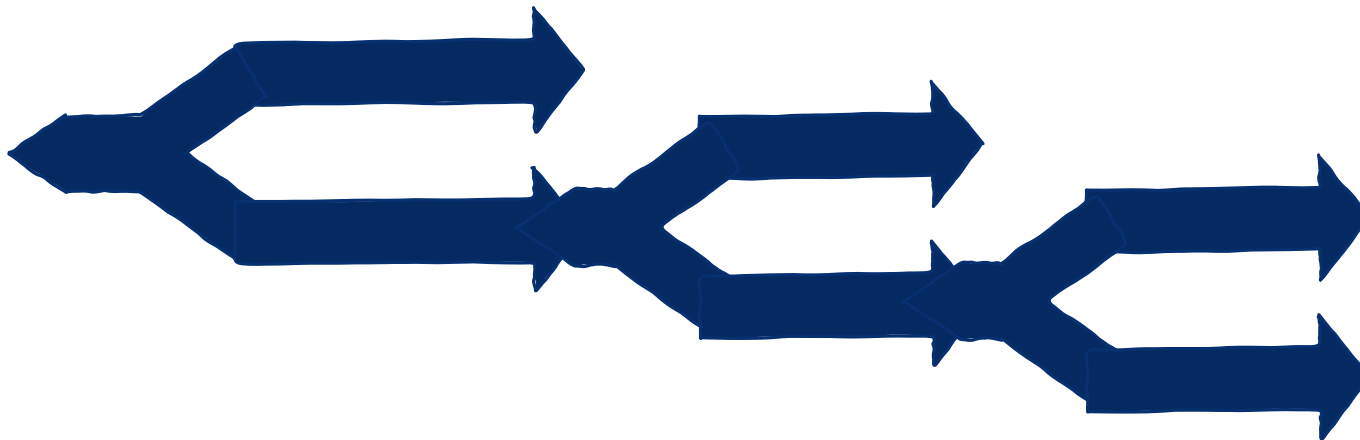
Cost $\uparrow$, delays between stages, flushing the pipeline because of branches

Dealing with Branches

- Multiple Streams
- Prefetch Branch Target
- Loop buffer
- Branch prediction
- Delayed branching

Multiple Streams

- Have two pipelines
- *Prefetch each branch* into a separate pipeline
- Use appropriate pipeline
- Leads to bus & register *contention*
- Multiple branches lead to *further pipelines* being needed



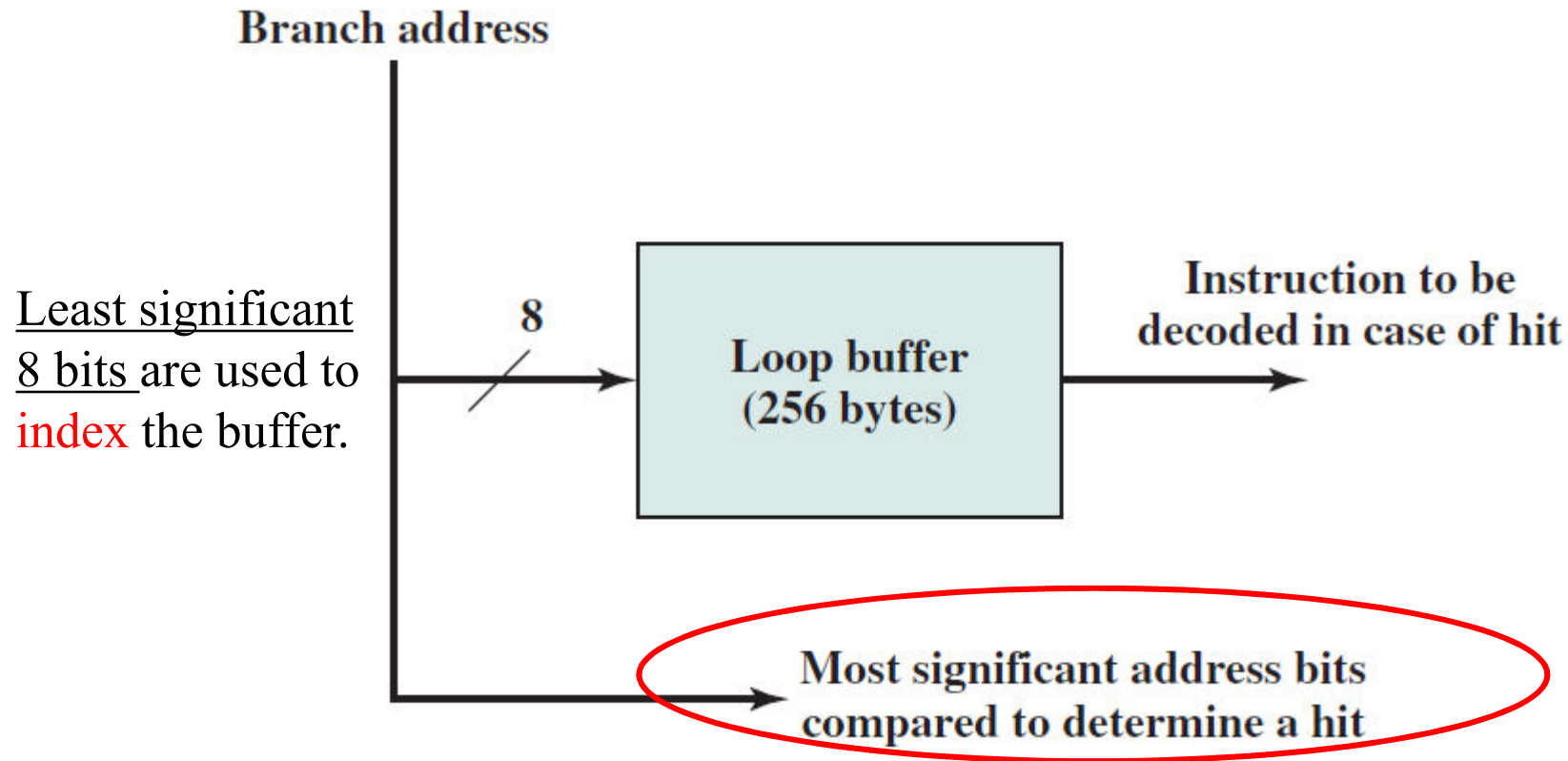
Prefetch Branch Target

- When a conditional branch is recognized, target of branch is *prefetched* in addition to instructions following branch
- Keep target until branch is executed
- Used by IBM 360/91

Loop Buffer

- A small, very fast memory **like cache**
- Containing the *n most recently fetched* instructions
- Maintained by *fetch stage* of pipeline
- Check buffer before fetching from memory
- Very good for *small loops* or jumps

Loop Buffer Diagram



	Cache	Loop buffer
size	Big	Much smaller
contents	Data/Instructions	Instructions

Branch Prediction

- Static prediction
 - Not related to branch history
- Dynamic prediction
 - Related to branch history

Static prediction

- Predict never taken
 - Assume that jump will not happen
 - *Always fetch next instruction*
 - 68020 & VAX 11/780
- Predict always taken
 - Assume that jump will happen
 - *Always fetch branch instruction*
 - May cause page fault
- Predict by Opcode
 - Some instructions are more likely to result in a jump than others
 - Can get up to 75% success

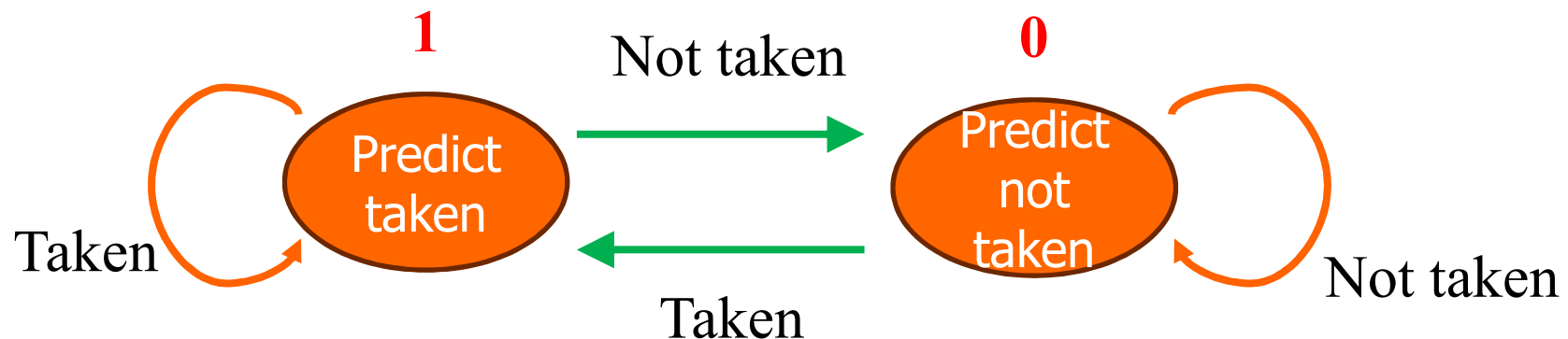
Dynamic Branch Prediction

- Taken/Not taken switch
 - Based on previous history bits(1 or more bits)
 - Good for loops
 - More than 80% accuracy
- Branch history table (Branch target buffer)
- Delayed Branch for RISC
 - Do not take jump until you have to
 - Rearrange instructions

Dynamic Branch Prediction

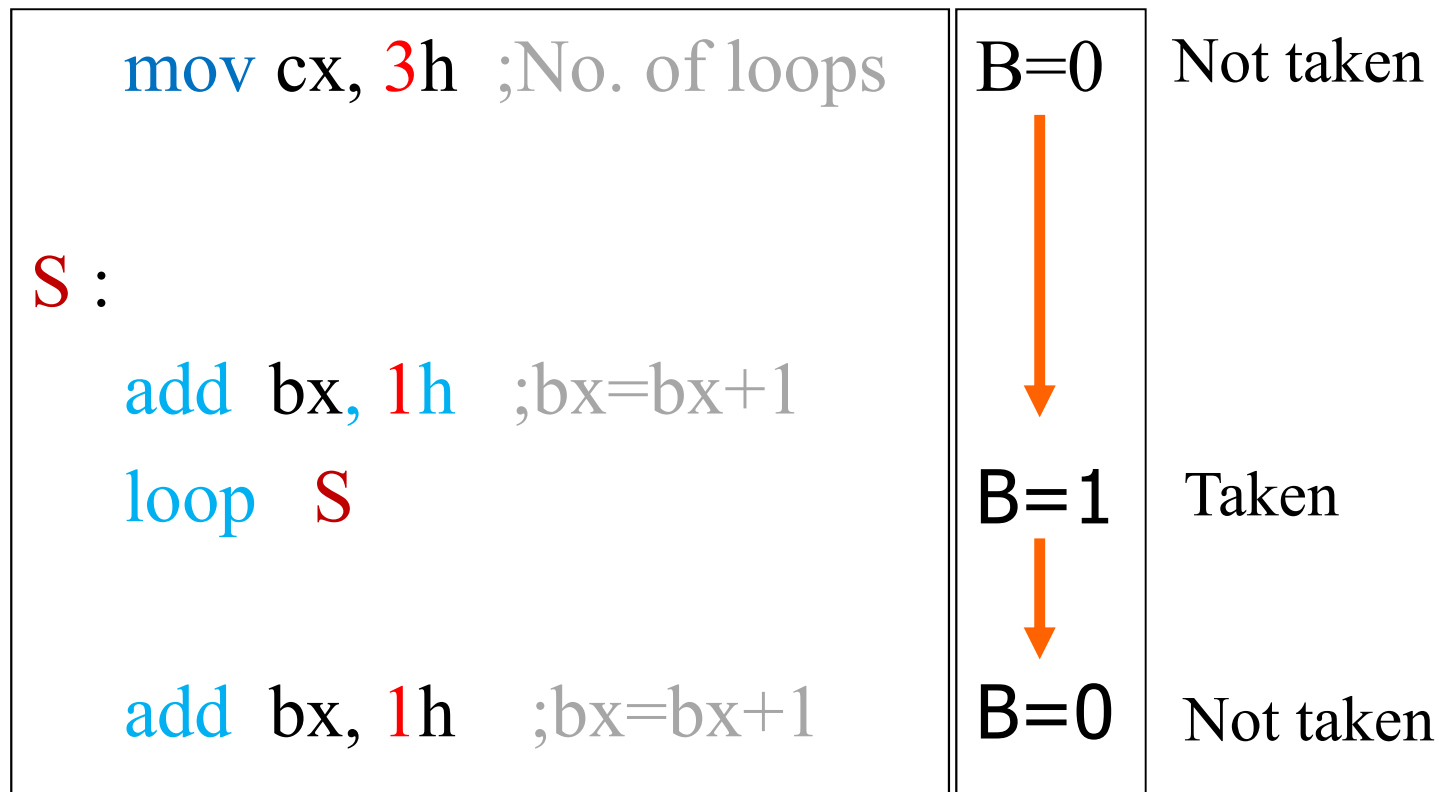
❑ Taken/Not taken switch

- **A Single bit:** record the last execution (branch or not)
- A **shortcoming** of using a single bit : almost always taken, such as a loop instruction.
- With only one bit of history, an error in prediction will occur **twice for each use of the loop**: once on entering the loop, and once on exiting.



Dynamic Branch Prediction

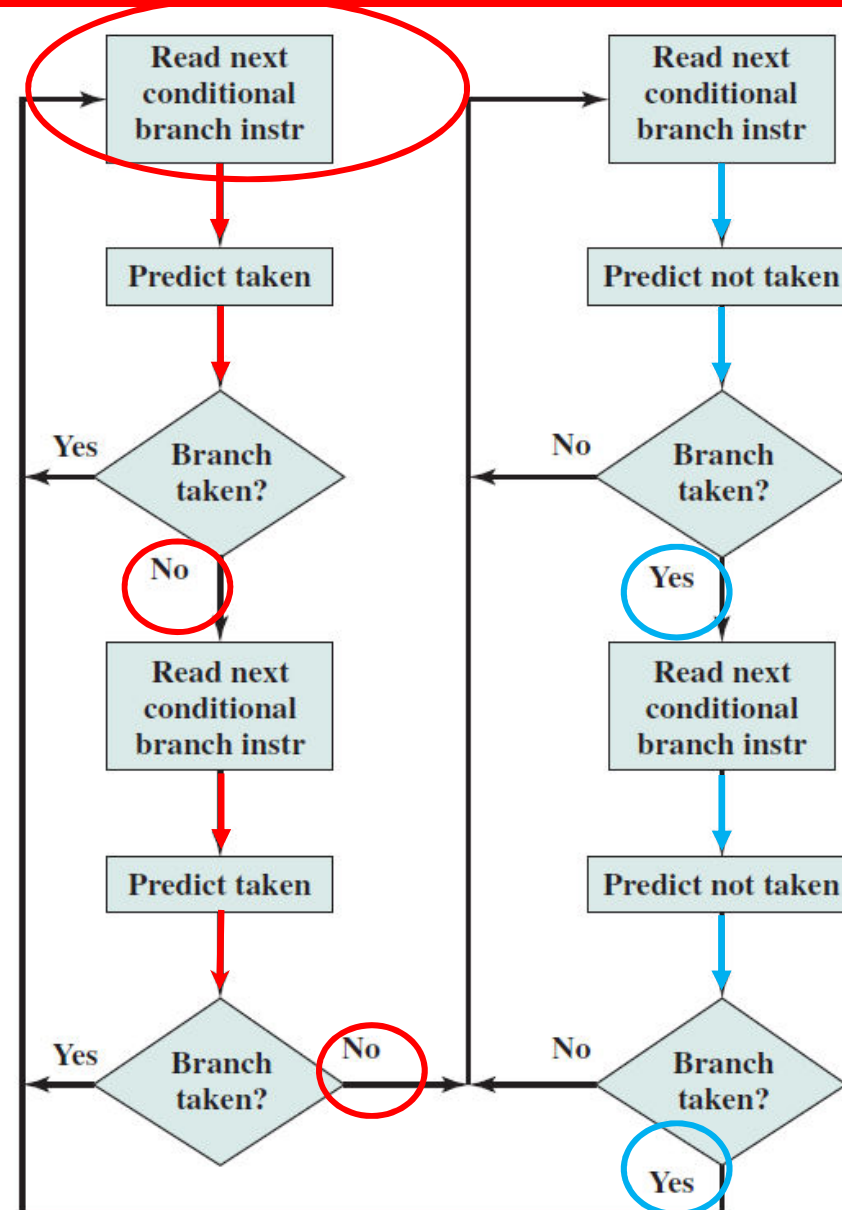
With only one bit of history, an error in prediction will occur **twice for each use of the loop**: once on entering the loop, and once on exiting



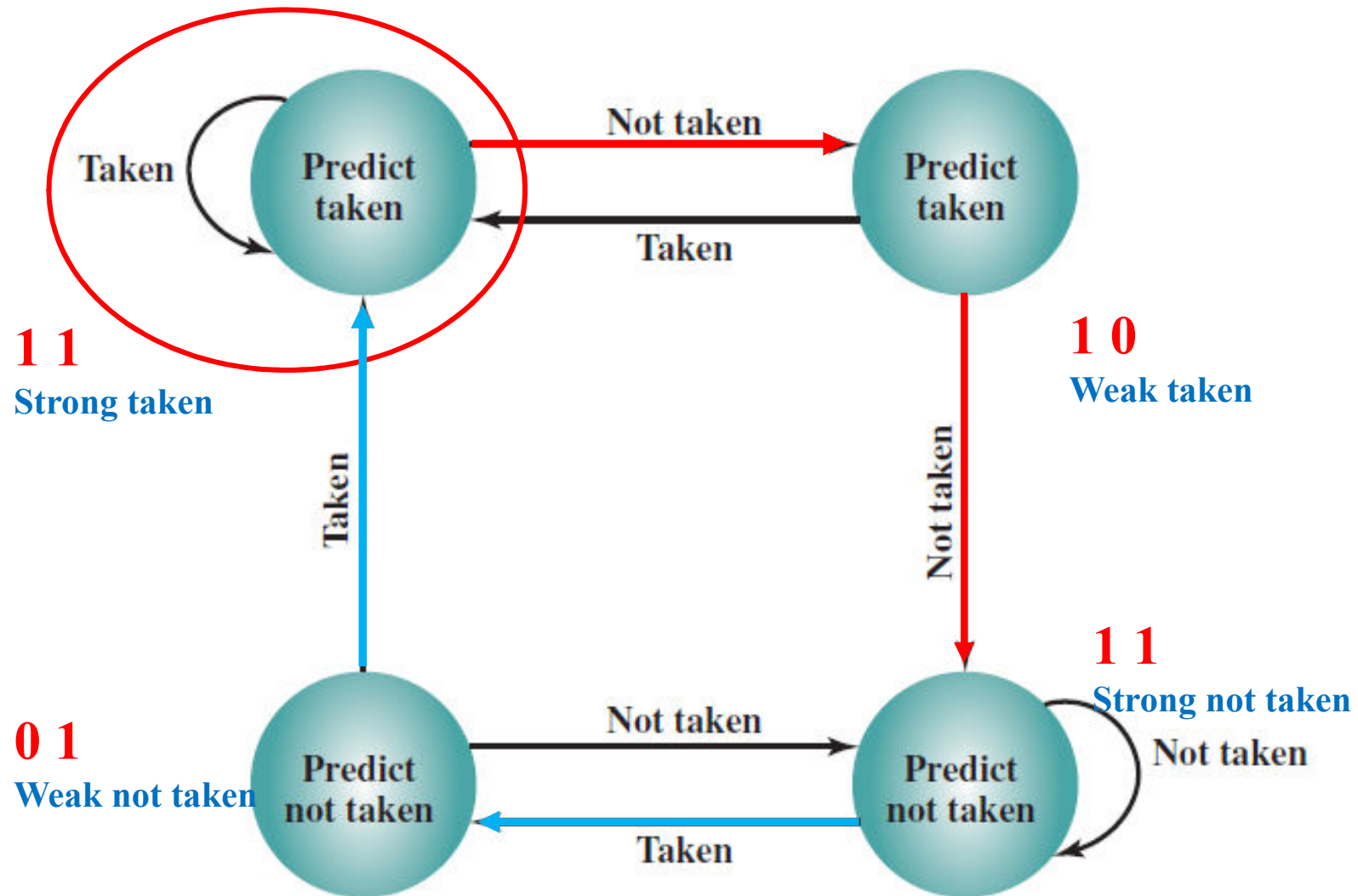
Branch Prediction Flowchart

Two bits:

The algorithm requires two consecutive wrong predictions to change the prediction decision.



Branch Prediction State Diagram



Branch target buffer/history table

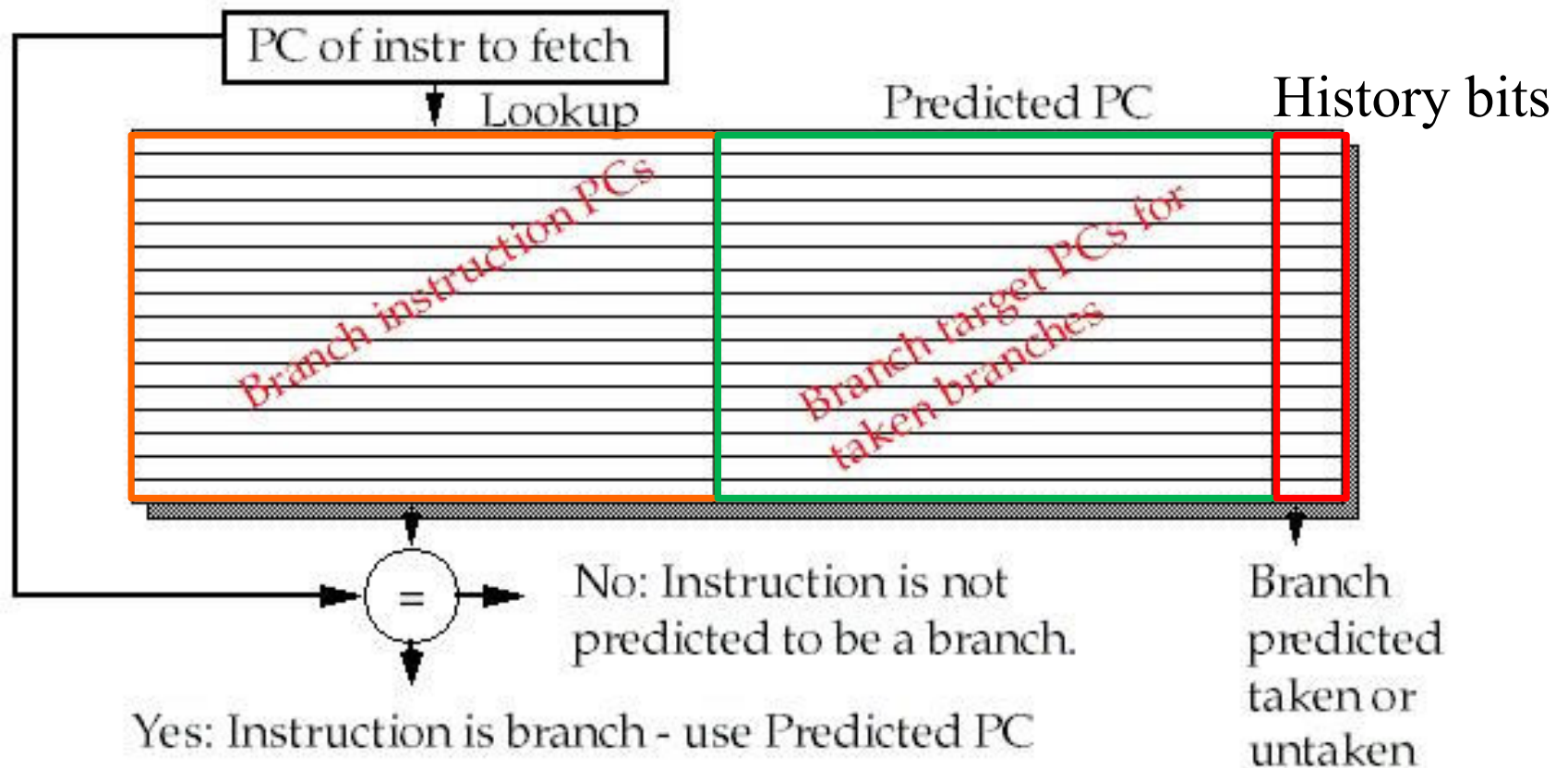
- The **drawbacks** of **history bits**: The target instruction cannot be fetched until the target address (**an operand**) is decoded.
- The branch history table is a small cache memory associated with the instruction fetch stage of the pipeline. Each entry in the table consists of three elements:



- 1. **Address of branch** instruction
- 2. History bits
- 3. Information about the target instruction (**target address** or target instruction)

Branch-Target Buffers

- *Branch target buffer structure:*

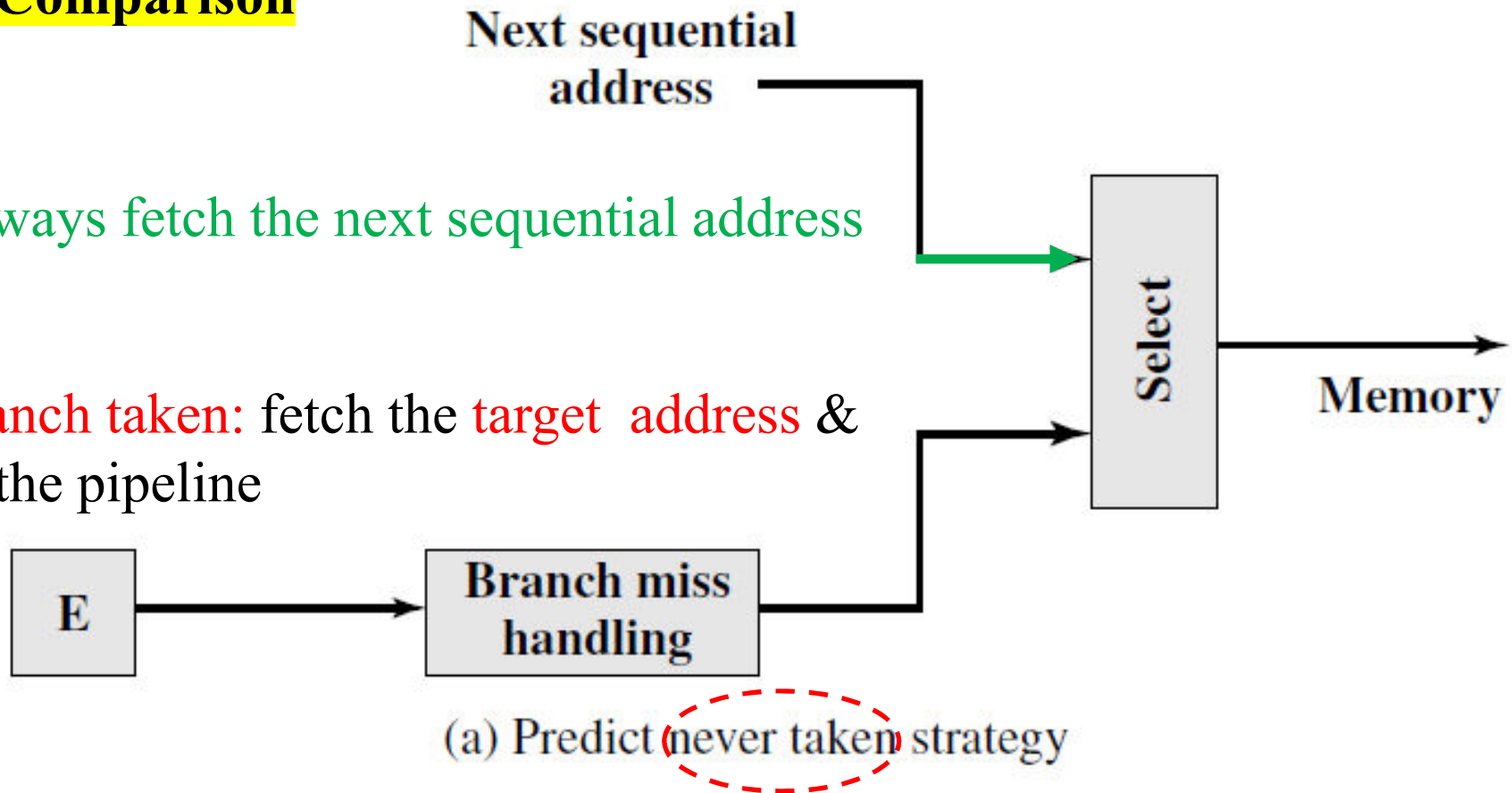


Branch target buffer/history table

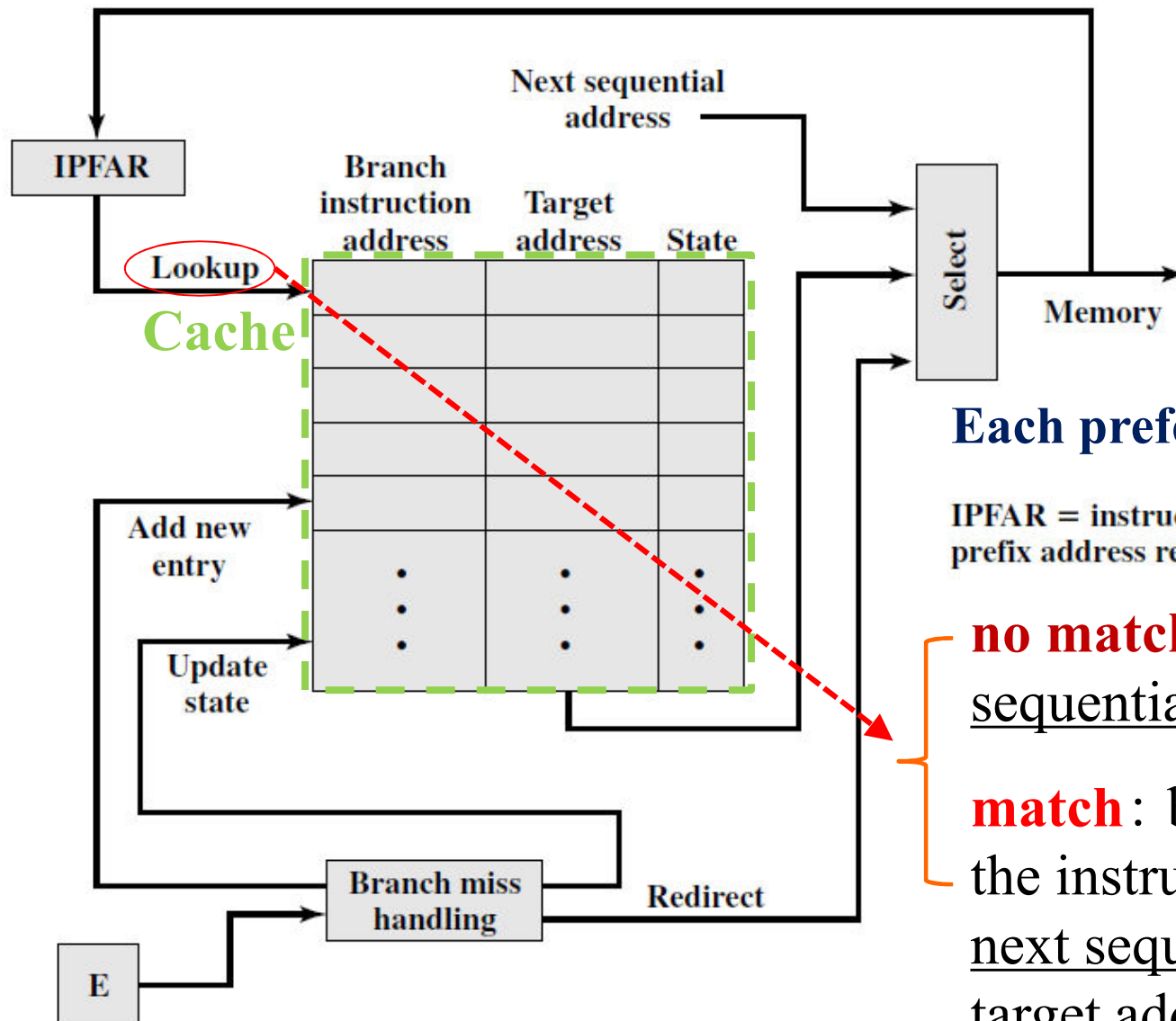
■ Comparison

1) Always fetch the next sequential address

2) Branch taken: fetch the target address & flush the pipeline



Branch target buffer/history table



Each prefetch triggers a lookup

IPFAR = instruction prefix address register

no match: choose the next sequential address

match: based on the **state** of the instruction. Choose the next sequential address or the target address

(b) Branch history table strategy

Homework

- Translate Key Terms
- Review Questions: 5,6,7
- Problems: 7,8,11